LAPORAN PRAKTIKUM PEMROGRAMAN WEB
MODUL 7: CRUD DAN UPLOAD FILE PADA LARAVEL



Disusun Oleh:

Nama: Andhika Yuridho

NIM: 707012500059

Kelas: D4SIKC-49-04

Mata Kuliah: Pemrograman Web

UNIVERSITAS TELKOM

FAKULTAS ILMU TERAPAN

JURUSAN S1 TERAPAN SISTEM INFORMASI KOTA CERDAS

TAHUN AJARAN 2026

1. Tujuan

Praktikum Modul 7 ini mengerjakan dua aplikasi CRUD yang sama-sama menyimpan berkas di folder storage, dengan dua cara membangun antarmuka yang berbeda. Tujuan yang ingin saya capai adalah sebagai berikut:

1. Memahami cara kerja resource controller dan resource route dalam menangani tujuh operasi CRUD.

2. Membangun fitur CRUD Berkas dengan Laravel Breeze sehingga hanya pengguna yang sudah login yang dapat mengelola berkas.

3. Menerapkan upload file melalui form multipart, validasi jenis dan ukuran berkas, serta penyimpanan ke disk public.

4. Menampilkan kembali berkas dari storage/app/public melalui symbolic link public/storage.

5. Mengganti dan menghapus berkas lama agar tidak ada file yang tertinggal di folder storage.

6. Membangun CRUD Kontak secara manual dengan layout Blade dan Bootstrap, termasuk authentication yang ditulis sendiri.

7. Membandingkan pengerjaan dengan Breeze dan tanpa Breeze pada bagian tampilan, authentication, dan pesan galat.

2. Alat dan Bahan

Praktikum ini dikerjakan pada satu laptop Lenovo 83DV dengan prosesor Intel Core i7-13650HX (14 core, 20 thread), RAM 12 GB, dan Windows 11 Pro versi 10.0.26100. Perangkat lunak yang dipakai adalah Laragon sebagai server lokal dengan Apache, MySQL 8.4.3, dan phpMyAdmin 5.2.3; PHP 8.3.30 (ZTS); Composer 2.10.3; Laravel Framework 13.38.0; Laravel Breeze 2.4; Node.js 24.22.1 dan npm 11.19.1; Vite 8.3.5 dengan Tailwind CSS 3.4.19 untuk tampilan Breeze; Bootstrap 5.3.0 dan Font Awesome 6.4.0 yang dimuat lewat CDN untuk versi manual; Visual Studio Code 1.143.1; Git 2.51.2; serta Google Chrome untuk pengujian.

Kedua project memakai database MySQL yang terpisah, yaitu db_crud_breeze untuk versi Breeze dan db_crud_manual untuk versi manual. Berkas yang dipakai sebagai bahan uji adalah gambar JPG dan PNG, dokumen PDF, arsip ZIP, dan satu gambar berukuran 7,5 MB untuk menguji batas ukuran unggahan.

3. Langkah-Langkah Praktikum

Pengerjaan dibagi menjadi dua bagian sesuai materi 7.4.4 dan 7.4.5. Bagian pertama membangun CRUD Berkas di atas Laravel Breeze, bagian kedua membangun CRUD Kontak dari project kosong dengan layout Bootstrap dan authentication sendiri. Seluruh perintah dijalankan dari terminal di dalam folder project masing-masing.

3.1 Menyiapkan Project dan Memasang Laravel Breeze

Project pertama dibuat dengan perintah composer create-project laravel/laravel praktikum7-jurnal. Nama foldernya saya sesuaikan menjadi praktikum7-jurnal, sedangkan nama database tetap db_crud_breeze seperti pada modul. Setelah project terbentuk, Breeze dipasang melalui composer require laravel/breeze --dev lalu php artisan breeze:install blade. Perintah kedua menyalin route, controller, dan view untuk register, login, logout, lupa password, dan halaman profil, sekaligus membuat file routes/auth.php.

Konfigurasi database di file .env diubah menjadi DB_CONNECTION=mysql, DB_HOST=127.0.0.1, DB_PORT=3306, DB_DATABASE=db_crud_breeze, DB_USERNAME=root, dan DB_PASSWORD kosong. Setelah itu saya menjalankan npm install dan npm run build, lalu php artisan migrate. Saya memakai npm run build, bukan npm run dev, karena asset hanya perlu dikompilasi sekali ke folder public/build dan tidak ada perubahan tampilan yang dikerjakan sambil server Vite dibiarkan berjalan. Terakhir php artisan storage:link dijalankan agar folder public/storage mengarah ke storage/app/public.

Server dijalankan dengan php artisan serve --port=8123, lalu satu akun didaftarkan melalui halaman /register. Akun ini yang dipakai untuk menguji seluruh fitur CRUD Berkas.

3.2 Membuat Migration dan Model Upload

Tabel uploads dibuat melalui perintah php artisan make:model Upload -m. File migration yang terbentuk kemudian diisi dengan kolom user_id sebagai pemilik berkas, judul, keterangan, filename, dan filepath.

```php
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    /**
     * Run the migrations.
     */
    public function up(): void
    {
        Schema::create('uploads', function (Blueprint $table) {
            $table->id();
            $table->foreignId('user_id')->constrained()->cascadeOnDelete();
            $table->string('judul');
            $table->text('keterangan')->nullable();
            $table->string('filename');
            $table->string('filepath');
            $table->timestamps();
        });
    }

    /**
     * Reverse the migrations.
     */
    public function down(): void
    {
        Schema::dropIfExists('uploads');
    }
};

```

Kolom user_id memakai foreignId()->constrained()->cascadeOnDelete() sehingga relasinya terhubung ke tabel users dan data berkas ikut terhapus ketika akun pemiliknya dihapus. Kolom keterangan diberi nullable() karena boleh dikosongkan, sedangkan filename menyimpan nama asli berkas dan filepath menyimpan lokasi hasil store(). Setelah migration dijalankan dengan php artisan migrate, tabel uploads muncul di database db_crud_breeze.

Model Upload diisi dengan properti $table, $fillable, dan relasi belongsTo ke model User.

```php
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Model;

class Upload extends Model
{
    protected $table = 'uploads';

    protected $fillable = [
        'user_id',
        'judul',
        'keterangan',
        'filename',
        'filepath'
    ];

    public function user()
    {
        return $this->belongsTo(User::class);
    }
}

```

Daftar $fillable menentukan kolom yang boleh diisi lewat Upload::create(), sehingga user_id, judul, keterangan, filename, dan filepath bisa disimpan sekaligus dalam satu pemanggilan. Relasi user() dipakai pada halaman detail untuk menampilkan nama pemilik berkas.

3.3 Membuat Resource Controller UploadController

Controller dibuat dengan php artisan make:controller UploadController --resource. Bagian pertama berisi method index, create, dan store.

```php
<?php

namespace App\Http\Controllers;

use App\Models\Upload;
use Illuminate\Http\Request;
use Illuminate\Support\Facades\Auth;
use Illuminate\Support\Facades\Storage;

class UploadController extends Controller
{
    /**
     * Display a listing of the resource.
     */
    public function index()
    {
        $uploads = Upload::where('user_id', Auth::id())->latest()->paginate(5);
        return view('uploads.index', compact('uploads'));
    }

    /**
     * Show the form for creating a new resource.
     */
    public function create()
    {
        return view('uploads.create');
    }

    /**
     * Store a newly created resource in storage.
     */
    public function store(Request $request)
    {
        $request->validate([
            'judul' => 'required|string|max:255',
            'keterangan' => 'nullable|string',
            'file' => 'required|file|mimes:jpg,jpeg,png,pdf,doc,docx|max:2048'
        ]);

        $file = $request->file('file');
        $path = $file->store('uploads', 'public');

        Upload::create([
            'user_id' => Auth::id(),
            'judul' => $request->judul,
            'keterangan' => $request->keterangan,
            'filename' => $file->getClientOriginalName(),
            'filepath' => $path
        ]);

        return redirect()->route('uploads.index')->with('success', 'Berkas berhasil
diunggah.');
    }

```

Method index hanya mengambil berkas milik pengguna yang sedang login melalui Upload::where('user_id', Auth::id())->latest()->paginate(5), sehingga daftar berisi paling banyak lima data per halaman dan diurutkan dari yang terbaru. Method store memvalidasi

judul, keterangan, dan file, lalu menyimpan berkas dengan $file->store('uploads', 'public'). Path hasil store() dan nama asli berkas dari getClientOriginalName() ikut disimpan ke database.

Bagian kedua berisi method show, edit, update, dan destroy.

```php
    /**
     * Display the specified resource.
     */
    public function show(Upload $upload)
    {
        abort_if($upload->user_id !== Auth::id(), 403);
        return view('uploads.show', compact('upload'));
    }

    /**
     * Show the form for editing the specified resource.
     */
    public function edit(Upload $upload)
    {
        abort_if($upload->user_id !== Auth::id(), 403);
        return view('uploads.edit', compact('upload'));
    }

    /**
     * Update the specified resource in storage.
     */
    public function update(Request $request, Upload $upload)
    {
        abort_if($upload->user_id !== Auth::id(), 403);

        $request->validate([
            'judul' => 'required|string|max:255',
            'keterangan' => 'nullable|string',
            'file' => 'nullable|file|mimes:jpg,jpeg,png,pdf,doc,docx|max:2048'
        ]);

        $data = $request->only('judul', 'keterangan');

        if ($request->hasFile('file')) {
            Storage::disk('public')->delete($upload->filepath);
            $file = $request->file('file');
            $data['filename'] = $file->getClientOriginalName();
            $data['filepath'] = $file->store('uploads', 'public');
        }

        $upload->update($data);

        return redirect()->route('uploads.index')->with('success', 'Berkas berhasil
diperbarui.');
    }

    /**
     * Remove the specified resource from storage.
     */
    public function destroy(Upload $upload)
    {
        abort_if($upload->user_id !== Auth::id(), 403);

        Storage::disk('public')->delete($upload->filepath);
        $upload->delete();

        return redirect()->route('uploads.index')->with('success', 'Berkas berhasil
dihapus.');
    }
}

```

Setiap method yang menerima Upload $upload memanggil abort_if($upload->user_id !== Auth::id(), 403) lebih dulu, sehingga berkas milik pengguna lain tidak bisa dibuka, diubah, maupun dihapus. Pada method update, berkas baru hanya diproses jika $request-

>hasFile('file') bernilai benar. Berkas lama dihapus dengan Storage::disk('public')->delete() sebelum path baru disimpan. Method destroy menghapus file fisik lalu menghapus barisnya dari tabel uploads.

3.4 Mendefinisikan Route dan Menu Berkas

Route resource uploads diletakkan di dalam grup middleware auth pada file routes/web.php, sehingga seluruh halaman CRUD Berkas hanya bisa diakses setelah login.

```php
<?php

use App\Http\Controllers\ProfileController;
use App\Http\Controllers\UploadController;
use Illuminate\Support\Facades\Route;

Route::get('/', function () {
    return view('welcome');
});

Route::get('/dashboard', function () {
    return view('dashboard');
})->middleware(['auth', 'verified'])->name('dashboard');

Route::middleware('auth')->group(function () {
    Route::get('/profile', [ProfileController::class, 'edit'])->name('profile.edit');
    Route::patch('/profile', [ProfileController::class, 'update'])->name('profile.update');
    Route::delete('/profile', [ProfileController::class, 'destroy'])->name('profile.destroy');
    Route::resource('uploads', UploadController::class);
});

require __DIR__ . '/auth.php';

```

Satu baris Route::resource('uploads', UploadController::class) menghasilkan tujuh route, yaitu uploads.index, uploads.create, uploads.store, uploads.show, uploads.edit, uploads.update, dan uploads.destroy. Daftarnya saya periksa dengan php artisan route:list --name=uploads. Menu Berkas ditambahkan pada navigasi Breeze dengan komponen x-nav-link.

```
                    <x-nav-link :href="route('dashboard')" :active="request()->routeIs('dashboard')">
                        {{ __('Dashboard') }}
                    </x-nav-link>
                    <x-nav-link :href="route('uploads.index')" :active="request()->routeIs('uploads.*')">
                        {{ __('Berkas') }}
                    </x-nav-link>

```

Atribut :active="request()->routeIs('uploads.*')" membuat menu Berkas bergaris bawah ketika pengguna sedang membuka salah satu halaman uploads. Komponen yang sama juga ditambahkan pada bagian navigasi versi mobile.

3.5 Membuat View CRUD Berkas

Ada empat file view di folder resources/views/uploads. File index.blade.php menampilkan daftar berkas beserta pesan berhasil dan tombol pagination. Bagian atas file berisi layout, pesan session, dan tombol unggah.

```
<x-app-layout>
    <x-slot name="header">
        <h2 class="font-semibold text-xl text-gray-800 leading-tight">
            Daftar Berkas
        </h2>
    </x-slot>

    <div class="py-12">
        <div class="max-w-7xl mx-auto sm:px-6 lg:px-8">
            <div class="bg-white shadow-sm sm:rounded-lg p-6">
                @if (session('success'))
                <div class="mb-4 p-3 rounded bg-green-100 text-green-700">
                    {{session('success')}}
                </div>
                @endif

                <a href="{{route('uploads.create')}}" class="inline-block mb-4 px-4 py-2 bg-blue-600 text-white rounded">+ Unggah Berkas</a>

```

Bagian bawahnya berisi tabel data dengan @forelse, tombol aksi, dan pesan ketika data kosong.

```blade
                <table class="w-full border text-left">
                    <thead class="bg-gray-100">
                        <tr>
                            <th class="border p-2">No</th>
                            <th class="border p-2">Judul</th>
                            <th class="border p-2">Nama File</th>
                            <th class="border p-2">Tanggal Unggah</th>
                            <th class="border p-2">Aksi</th>
                        </tr>
                    </thead>
                    <tbody>
                        @forelse ($uploads as $upload)
                            <tr>
                                <td class="border p-2">{{ $uploads->firstItem() + $loop->index }}</td>
                                <td class="border p-2">{{ $upload->judul }}</td>
                                <td class="border p-2">
                                    <a href="{{ asset('storage/' . $upload->filepath) }}" target="_blank"
                                       class="text-blue-600 underline">{{ $upload->filename }}</a>
                                </td>
                                <td class="border p-2">{{ $upload->created_at->format('d-m-Y H:i') }}</td>
                                <td class="border p-2">
                                    <a href="{{ route('uploads.show', $upload) }}" class="text-sky-600">Lihat</a> |
                                    <a href="{{ route('uploads.edit', $upload) }}" class="text-yellow-600">Edit</a> |
                                    <form action="{{ route('uploads.destroy', $upload) }}" method="POST" class="inline">
                                        @csrf
                                        @method('DELETE')
                                        <button type="submit" class="text-red-600"
                                                onclick="return confirm('Yakin ingin menghapus berkas ini?')">Hapus</button>
                                    </form>
                                </td>
                            </tr>
                        @empty
                            <tr>
                                <td colspan="5" class="border p-2 text-center">Belum ada berkas yang diunggah.</td>
                            </tr>
                        @endforelse
                    </tbody>
                </table>

                <div class="mt-4">{{ $uploads->links() }}</div>
            </div>
        </div>
    </div>
</x-app-layout>

```

Nomor urut memakai $uploads->firstItem() + $loop->index agar penomoran berlanjut di halaman berikutnya. Setiap baris memiliki tautan Lihat, Edit, dan tombol Hapus yang dikirim sebagai request DELETE melalui @method('DELETE').

File create.blade.php memuat form unggah dengan enctype multipart/form-data.

```
<x-app-layout>
    <x-slot name="header">
        <h2 class="font-semibold text-xl text-gray-800 leading-tight">Unggah Berkas</h2>
    </x-slot>
    <div class="py-12">
        <div class="max-w-3xl mx-auto sm:px-6 lg:px-8">
            <div class="bg-white shadow-sm sm:rounded-lg p-6">
                {{-- enctype wajib ada agar file ikut terkirim --}}
                <form action="{{ route('uploads.store') }}" method="POST"
enctype="multipart/form-data">
                    @csrf
                    <div class="mb-4">
                        <x-input-label for="judul" value="Judul" />
                        <x-text-input id="judul" name="judul" type="text" class="mt-1
block w-full" :value="old('judul')" required autofocus />
                        <x-input-error :messages="$errors->get('judul')" class="mt-2" />
                    </div>

                    <div class="mb-4">
                        <x-input-label for="keterangan" value="Keterangan (opsional)" />
                        <textarea id="keterangan" name="keterangan" rows="3" class="mt-1
block w-full border-gray-300 rounded-md shadow-sm">{{ old('keterangan') }}</textarea>
                        <x-input-error :messages="$errors->get('keterangan')" class="mt-2"
/>
                    </div>

                    <div class="mb-4">
                        <x-input-label for="file" value="Pilih Berkas (jpg, jpeg, png,
pdf, doc, docx - maks. 2 MB)" />
                        <input id="file" name="file" type="file" class="mt-1 block w-full"
required>
                        <x-input-error :messages="$errors->get('file')" class="mt-2" />
                    </div>

                    <x-primary-button>Simpan</x-primary-button>
                    <a href="{{ route('uploads.index') }}" class="ms-3 text-gray-
600">Batal</a>
                </form>
            </div>
        </div>
    </div>
</x-app-layout>

```

Atribut enctype wajib ada pada form yang mengirim berkas. Tanpa atribut tersebut, isi file tidak ikut terkirim dan validasi file akan selalu gagal. Komponen x-input-error menampilkan pesan galat validasi tepat di bawah input yang bermasalah.

File edit.blade.php memakai form yang sama dengan tambahan @method('PUT') dan tautan ke berkas yang sedang tersimpan.

```blade
<x-app-layout>
    <x-slot name="header">
        <h2 class="font-semibold text-xl text-gray-800 leading-tight">Edit Berkas</h2>
    </x-slot>

    <div class="py-12">
        <div class="max-w-3xl mx-auto sm:px-6 lg:px-8">
            <div class="bg-white shadow-sm sm:rounded-lg p-6">
                <form action="{{ route('uploads.update', $upload) }}" method="POST" enctype="multipart/form-data">
                    @csrf
                    @method('PUT')
                    <div class="mb-4">
                        <x-input-label for="judul" value="Judul" />
                        <x-text-input id="judul" name="judul" type="text" class="mt-1 block w-full" :value="old('judul', $upload->judul)" required />
                        <x-input-error :messages="$errors->get('judul')" class="mt-2" />
                    </div>

                    <div class="mb-4">
                        <x-input-label for="keterangan" value="Keterangan (opsional)" />
                        <textarea id="keterangan" name="keterangan" rows="3" class="mt-1 block w-full border-gray-300 rounded-md shadow-sm">{{ old('keterangan', $upload->keterangan) }}</textarea>
                    </div>

                    <div class="mb-4">
                        <p class="text-sm text-gray-600">File saat ini:
                            <a href="{{ asset('storage/' . $upload->filepath) }}" target="_blank"
                                class="text-blue-600 underline">{{ $upload->filename }}</a>

                        </p>
                        <x-input-label for="file" value="Ganti Berkas (kosongkan jika tidak ingin mengganti)" class="mt-2" />
                        <input id="file" name="file" type="file" class="mt-1 block w-full">
                        <x-input-error :messages="$errors->get('file')" class="mt-2" />
                    </div>

                    <x-primary-button>Update</x-primary-button>
                    <a href="{{ route('uploads.index') }}" class="ms-3 text-gray-600">Batal</a>
                </form>
            </div>
        </div>
    </div>
</x-app-layout>

```

Input file pada form edit tidak diberi atribut required karena penggantian berkas bersifat opsional. Nilai old('judul', $upload->judul) membuat input terisi data lama, atau data yang baru diketik jika validasi gagal.

File show.blade.php menampilkan detail berkas dan pratinjau sesuai jenis filenya.

```blade
<x-app-layout>
    <x-slot name="header">
        <h2 class="font-semibold text-xl text-gray-800 leading-tight">Detail Berkas</h2>
    </x-slot>

    <div class="py-12">
        <div class="max-w-3xl mx-auto sm:px-6 lg:px-8">
            <div class="bg-white shadow-sm sm:rounded-lg p-6 space-y-2">
                <p><strong>Judul:</strong> {{ $upload->judul }}</p>
                <p><strong>Keterangan:</strong> {{ $upload->keterangan ?? '-' }}</p>
                <p><strong>Nama File:</strong> {{ $upload->filename }}</p>
                <p><strong>Diunggah oleh:</strong> {{ $upload->user->name }}</p>
                <p><strong>Tanggal Unggah:</strong> {{ $upload->created_at->format('d-m-Y H:i') }}</p>

                @php
                    $ext = strtolower(pathinfo($upload->filepath, PATHINFO_EXTENSION));
                @endphp

                {{-- Jika file berupa gambar, tampilkan pratinjau --}}
                @if (in_array($ext, ['jpg', 'jpeg', 'png']))
                    <img src="{{ asset('storage/' . $upload->filepath) }}" alt="{{ $upload->judul }}" class="max-w-sm rounded border">
                @else
                    <a href="{{ asset('storage/' . $upload->filepath) }}" target="_blank" class="inline-block px-4 py-2 bg-gray-800 text-white rounded">Buka / Unduh Berkas</a>
                @endif

                <div class="pt-4">
                    <a href="{{ route('uploads.edit', $upload) }}" class="text-yellow-600">Edit</a> |
                    <a href="{{ route('uploads.index') }}" class="text-gray-600">Kembali</a>
                </div>
            </div>
        </div>
    </div>
</x-app-layout>

```

Ekstensi berkas diambil dengan pathinfo($upload->filepath, PATHINFO_EXTENSION). Jika ekstensinya jpg, jpeg, atau png, halaman menampilkan pratinjau gambar. Untuk berkas lain, halaman menampilkan tombol Buka / Unduh Berkas yang mengarah ke URL storage.

3.6 Menyiapkan Project Manual dan Layout Bootstrap

Project kedua dibuat dengan composer create-project laravel/laravel praktikum7-jurnal-manual-tanpa-breeze dan memakai database db_crud_manual. Breeze tidak dipasang pada project ini. Tampilan dibangun dari layout Blade sendiri di resources/views/layouts/app.blade.php dengan Bootstrap 5.3.0 dan Font Awesome 6.4.0 dari CDN, sehingga tidak perlu Node.js maupun proses build asset.

```
<!DOCTYPE html>
<html lang="id">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>@yield('title', 'CRUD Kontak')</title>
    <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css"
rel="stylesheet">
    <link rel="stylesheet" href="https://cdnjs.cloudflare.com/ajax/libs/font-
awesome/6.4.0/css/all.min.css">
</head>
<body>
    {{-- Navbar (menu bagian atas website) --}}
     <nav class="navbar navbar-expand-lg navbar-dark bg-primary mb-4">


```

Bagian isi layout menampung pesan berhasil, ringkasan galat validasi, dan konten tiap halaman.

```blade
    <div class="container">
            <a class="navbar-brand" href="{{ route('kontaks.index') }}">CRUD Kontak</a>
            <ul class="navbar-nav ms-auto align-items-center">
                @auth
                    {{-- Menu untuk pengguna yang sudah login --}}
                    <li class="nav-item">
                        <a class="nav-link" href="{{ route('kontaks.index') }}">
                            <i class="fas fa-address-book me-1"></i>Kontak
                        </a>
                    </li>
                    <li class="nav-item">
                        <span class="nav-link text-white"><i class="fas fa-user me-1"></i>
{{ Auth::user()->name }}</span>
                    </li>
                    <li class="nav-item">
                        <form action="{{ route('logout') }}" method="POST">
                            @csrf
                            <button type="submit" class="btn btn-light btn-sm ms-2">

                                <i class="fas fa-sign-out-alt me-1"></i>Logout
                            </button>
                        </form>
                    </li>
                @else
                    {{-- Menu untuk tamu (belum login) --}}
                    <li class="nav-item">
                        <a class="nav-link" href="{{ route('login') }}"><i class="fas fa-
sign-in-alt me-1"></i>Login</a>
                    </li>
                    <li class="nav-item">
                        <a class="nav-link" href="{{ route('register') }}"><i class="fas
fa-user-plus me-1"></i>Register</a>
                    </li>
                @endauth
            </ul>
        </div>
    </nav>

    <div class="container">
        @if (session('success'))
            <div class="alert alert-success alert-dismissible fade show" role="alert">
                {{ session('success') }}
                <button type="button" class="btn-close" data-bs-dismiss="alert" aria-
label="Close"></button>
            </div>
        @endif

        @if ($errors->any())
            <div class="alert alert-danger">
                Data belum valid, periksa kembali isian form.
            </div>
        @endif
        @yield('content')
    </div>

    <script
src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/js/bootstrap.bundle.min.js">
</script>
</body>
</html>

```

Pesan berhasil ditampilkan dari session('success') yang dikirim controller melalui with(), sedangkan $errors->any() dipakai untuk menampilkan ringkasan galat. Navbar menampilkan

menu Kontak, nama pengguna, dan tombol Logout untuk pengguna yang sudah login, atau menu Login dan Register untuk tamu.

3.7 Membuat Migration dan Model Kontak

Tabel kontaks dibuat dengan php artisan make:model Kontak -m. Kolomnya adalah nama, alamat, no_hp, dan foto.

```php
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    public function up(): void
    {
        Schema::create('kontaks', function (Blueprint $table) {
            $table->id();                           // Primary key auto increment
            $table->string('nama');                 // VARCHAR(255)
            $table->text('alamat');                 // TEXT
            $table->string('no_hp', 15);            // Disimpan sebagai teks agar angka 0 di depan tidak hilang
            $table->string('foto')->nullable();     // Path foto, boleh kosong
            $table->timestamps();                   // created_at dan updated_at
        });
    }

    public function down(): void
    {
        Schema::dropIfExists('kontaks');
    }
};

```

Kolom no_hp memakai string dengan panjang 15 karakter agar angka 0 di depan nomor telepon tidak hilang. Kolom foto diberi nullable() karena kontak boleh disimpan tanpa foto. Model Kontak hanya berisi nama tabel dan daftar $fillable.

```php
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Model;

class Kontak extends Model
{
    protected $table = 'kontaks'; // Nama tabel yang dihubungkan

    // Kolom yang boleh diisi melalui Kontak::create() dan update()
    protected $fillable = [
        'nama',
        'alamat',
        'no_hp',
        'foto',
    ];
}

```

Setelah migration dijalankan, php artisan storage:link dipanggil sekali agar foto di storage/app/public/foto_kontak bisa ditampilkan lewat asset('storage/...').

3.8 Membuat KontakController dan Resource Route

Controller dibuat dengan php artisan make:controller KontakController --resource. Bagian pertama berisi method index, create, dan store.

```php
<?php

namespace App\Http\Controllers;

use App\Models\Kontak;
use Illuminate\Http\Request;
use Illuminate\Support\Facades\Storage;

class KontakController extends Controller
{
    public function index()
    {
        $kontaks = Kontak::latest()->paginate(5); // 5 data per halaman
        return view('kontaks.index', compact('kontaks'));
    }

    public function create()
    {
        return view('kontaks.create');
    }

    public function store(Request $request)
    {
        $validated = $request->validate([
            'nama'   => 'required|string|max:255',
            'alamat' => 'required|string',
            'no_hp'  => 'required|string|max:15',
            'foto'   => 'nullable|image|mimes:jpg,jpeg,png|max:2048', // Foto opsional, maks. 2 MB
        ]);

        if ($request->hasFile('foto')) {
            // Simpan ke storage/app/public/foto_kontak, lalu simpan path-nya
            $validated['foto'] = $request->file('foto')->store('foto_kontak', 'public');
        }

        Kontak::create($validated);

        return redirect()->route('kontaks.index')->with('success', 'Kontak berhasil ditambahkan.');
    }

```

Hasil validasi disimpan pada variabel $validated sehingga hanya kolom yang lolos validasi yang masuk ke database. Foto hanya diproses jika $request->hasFile('foto') bernilai benar, lalu disimpan ke folder foto_kontak pada disk public.

Bagian kedua berisi method show, edit, update, dan destroy.

```php
    public function show(Kontak $kontak)
    {
        return view('kontaks.show', compact('kontak'));
    }

    public function edit(Kontak $kontak)
    {
        return view('kontaks.edit', compact('kontak'));
    }

    public function update(Request $request, Kontak $kontak)
    {
        $validated = $request->validate([
            'nama'   => 'required|string|max:255',
            'alamat' => 'required|string',
            'no_hp'  => 'required|string|max:15',
            'foto'   => 'nullable|image|mimes:jpg,jpeg,png|max:2048',
        ]);

        if ($request->hasFile('foto')) {
            if ($kontak->foto) {
                Storage::disk('public')->delete($kontak->foto); // Hapus foto lama
            }
            $validated['foto'] = $request->file('foto')->store('foto_kontak', 'public');
        }

        $kontak->update($validated);

        return redirect()->route('kontaks.index')->with(
            'success',
            'Kontak berhasil diperbarui.'
        );
    }

    public function destroy(Kontak $kontak)
    {
        if ($kontak->foto) {
            Storage::disk('public')->delete($kontak->foto); // Hapus file foto
        }
        $kontak->delete(); // Hapus data dari database

        return redirect()->route('kontaks.index')->with('success', 'Kontak berhasil
dihapus.');
    }
}

```

Pada method update, foto lama dihapus dengan Storage::disk('public')->delete() hanya ketika pengguna memilih foto baru. Method destroy menghapus foto lebih dulu, lalu menghapus baris kontak dari database.

Route kontak didaftarkan sebagai resource dan halaman pengelolaannya dilindungi middleware auth.

```php
<?php

use App\Http\Controllers\AuthController;
use App\Http\Controllers\KontakController;
use Illuminate\Support\Facades\Route;

Route::get('/', function () {
    return redirect()->route('kontaks.index'); // Halaman awal langsung ke daftar kontak
});

// Halaman register dan login hanya untuk pengguna yang belum login
Route::middleware('guest')->group(function () {
    Route::get('/register', [AuthController::class, 'showRegister'])->name('register');
    Route::post('/register', [AuthController::class, 'register']);
    Route::get('/login', [AuthController::class, 'showLogin'])->name('login');
    Route::post('/login', [AuthController::class, 'login']);
});

// Halaman yang hanya dapat diakses setelah login
Route::middleware('auth')->group(function () {
    Route::post('/logout', [AuthController::class, 'logout'])->name('logout');

    // CRUD Kontak dengan upload foto
    Route::resource('kontaks', KontakController::class);
});

```

Halaman register, login, dan proses logout diletakkan di dalam grup middleware guest dan auth sesuai kebutuhannya. Route / langsung mengarah ke daftar kontak. Agar tombol pagination memakai gaya Bootstrap, Paginator::useBootstrapFive() ditambahkan pada method boot() di AppServiceProvider.

```php
<?php

namespace App\Providers;

use Illuminate\Pagination\Paginator;
use Illuminate\Support\ServiceProvider;

class AppServiceProvider extends ServiceProvider
{
    public function register(): void
    {
        //
    }

    public function boot(): void
    {
        Paginator::useBootstrapFive(); // Pagination menggunakan tampilan Bootstrap 5
    }
}

```

3.9 Membuat View CRUD Kontak

File index.blade.php menampilkan daftar kontak dengan thumbnail foto dan tombol aksi.

```
@extends('layouts.app')

@section('title', 'Daftar Kontak')

@section('content')
<div class="d-flex justify-content-between align-items-center mb-3">
    <h2>Daftar Kontak</h2>
    <a href="{{ route('kontaks.create') }}" class="btn btn-primary">
        <i class="fas fa-plus me-1"></i>Tambah Kontak
    </a>
</div>

```

Tabelnya memakai @forelse dengan pesan "Belum ada data kontak." ketika tabel masih kosong.

```blade
<table class="table table-bordered align-middle">
    <thead class="table-light">
        <tr>
            <th>No</th>
            <th>Foto</th>
            <th>Nama</th>
            <th>Alamat</th>
            <th>No HP</th>
            <th>Aksi</th>
        </tr>
    </thead>
    <tbody>
        @forelse ($kontaks as $kontak)
            <tr>
                <td>{{ $kontaks->firstItem() + $loop->index }}</td>
                <td>
                    @if ($kontak->foto)
                        <img src="{{ asset('storage/' . $kontak->foto) }}" alt="{{ $kontak->nama }}"
                             width="60" height="60" class="rounded object-fit-cover">
                    @else
                        <span class="text-muted">Tidak ada</span>
                    @endif
                </td>
                <td>{{ $kontak->nama }}</td>
                <td>{{ $kontak->alamat }}</td>
                <td>{{ $kontak->no_hp }}</td>
                <td>
                    <a href="{{ route('kontaks.show', $kontak) }}" class="btn btn-info btn-sm">Lihat</a>
                    <a href="{{ route('kontaks.edit', $kontak) }}" class="btn btn-warning btn-sm">Edit</a>
                    <form action="{{ route('kontaks.destroy', $kontak) }}" method="POST" class="d-inline">
                        @csrf
                        @method('DELETE')
                        <button type="submit" class="btn btn-danger btn-sm"
                                onclick="return confirm('Apakah Anda yakin ingin menghapus?')">Hapus</button>
                    </form>
                </td>
            </tr>
        @empty
            <tr>
                <td colspan="6" class="text-center">Belum ada data kontak.</td>
            </tr>
        @endforelse
    </tbody>
</table>

{{ $kontaks->links() }}
@endsection
```

Thumbnail hanya ditampilkan jika $kontak->foto tidak kosong, sedangkan kontak tanpa foto diberi teks Tidak ada. Tombol Hapus mengirim request DELETE melalui @method('DELETE') dan meminta konfirmasi lewat onclick.

File create.blade.php berisi form tambah kontak dengan input foto.

```
@extends('layouts.app')

@section('title', 'Tambah Kontak')

@section('content')
<div class="row justify-content-center">
    <div class="col-md-8">
        <div class="card">
            <div class="card-header">Tambah Kontak Baru</div>
            <div class="card-body">

                <form action="{{ route('kontaks.store') }}" method="POST"
enctype="multipart/form-data">
                    @csrf

                    <div class="mb-3">
                        <label for="nama" class="form-label">Nama</label>
                        <input type="text" id="nama" name="nama" value="{{ old('nama') }}"

                               class="form-control @error('nama') is-invalid @enderror"
required>
                        @error('nama') <div class="invalid-feedback">{{ $message }}</div>
@enderror
                    </div>

                    <div class="mb-3">
                        <label for="alamat" class="form-label">Alamat</label>
                        <textarea id="alamat" name="alamat" rows="3"
                                  class="form-control @error('alamat') is-invalid
@enderror" required>{{ old('alamat') }}</textarea>
                        @error('alamat') <div class="invalid-feedback">{{ $message }}
</div> @enderror
                    </div>

                    <div class="mb-3">
                        <label for="no_hp" class="form-label">No HP</label>
                        <input type="text" id="no_hp" name="no_hp" value="{{ old('no_hp')
}}"
                               class="form-control @error('no_hp') is-invalid @enderror"
required>
                        @error('no_hp') <div class="invalid-feedback">{{ $message }}</div>
@enderror
                    </div>

```

Bagian input foto memakai enctype multipart/form-data dan menampilkan pesan galat per input.

```
                    <div class="mb-3">
                        <label for="foto" class="form-label">Foto (jpg, jpeg, png - maks.
2 MB)</label>
                        <input type="file" id="foto" name="foto" accept="image/*"

                               class="form-control @error('foto') is-invalid @enderror">
                        @error('foto') <div class="invalid-feedback">{{ $message }}</div>
@enderror
                    </div>

                    <button type="submit" class="btn btn-primary">Simpan</button>
                    <a href="{{ route('kontaks.index') }}" class="btn btn-
secondary">Batal</a>
                </form>
            </div>
        </div>
    </div>
</div>
@endsection

```

File edit.blade.php menampilkan foto yang sedang tersimpan sebelum input penggantian foto.

```blade
@extends('layouts.app')

@section('title', 'Edit Kontak')

@section('content')
<div class="row justify-content-center">
    <div class="col-md-8">
        <div class="card">
            <div class="card-header">Edit Kontak</div>
            <div class="card-body">

                <form action="{{ route('kontaks.update', $kontak) }}" method="POST" enctype="multipart/form-data">
                    @csrf
                    @method('PUT')

                    <div class="mb-3">
                        <label for="nama" class="form-label">Nama</label>
                        <input type="text" id="nama" name="nama" value="{{ old('nama', $kontak->nama) }}"
                               class="form-control @error('nama') is-invalid @enderror" required>
                        @error('nama') <div class="invalid-feedback">{{ $message }}</div> @enderror
                    </div>

                    <div class="mb-3">
                        <label for="alamat" class="form-label">Alamat</label>
                        <textarea id="alamat" name="alamat" rows="3"
                                  class="form-control @error('alamat') is-invalid @enderror" required>{{ old('alamat', $kontak->alamat) }}</textarea>
                        @error('alamat') <div class="invalid-feedback">{{ $message }} </div> @enderror
                    </div>

                    <div class="mb-3">
                        <label for="no_hp" class="form-label">No HP</label>
                        <input type="text" id="no_hp" name="no_hp" value="{{ old('no_hp', $kontak->no_hp) }}"
                               class="form-control @error('no_hp') is-invalid @enderror" required>
                        @error('no_hp') <div class="invalid-feedback">{{ $message }}</div> @enderror
                    </div>

```

Bagian bawah form berisi keterangan bahwa input foto boleh dikosongkan, tombol Update, dan Batal.

```
                    <div class="mb-3">
                        <label class="form-label d-block">Foto Saat Ini</label>
                        @if ($kontak->foto)
                            <img src="{{ asset('storage/' . $kontak->foto) }}" alt="{{
$kontak->nama }}"
                                 width="120" class="rounded mb-2">
                        @else
                            <p class="text-muted">Belum ada foto.</p>
                        @endif
                        <input type="file" id="foto" name="foto" accept="image/*"
                               class="form-control @error('foto') is-invalid @enderror">
                        <small class="text-muted">Kosongkan jika tidak ingin mengganti
foto.</small>
                        @error('foto') <div class="invalid-feedback">{{ $message }}</div>
@enderror
                    </div>

                    <button type="submit" class="btn btn-primary">Update</button>
                    <a href="{{ route('kontaks.index') }}" class="btn btn-
secondary">Batal</a>
                </form>
            </div>
        </div>
    </div>
</div>
@endsection

```

File show.blade.php menampilkan satu kontak secara utuh beserta foto dan tombol aksi.

```blade
@extends('layouts.app')

@section('title', 'Detail Kontak')

@section('content')
<div class="row justify-content-center">
    <div class="col-md-8">
        <div class="card">
            <div class="card-header">Detail Kontak</div>
            <div class="card-body text-center">
                @if ($kontak->foto)
                    <img src="{{ asset('storage/' . $kontak->foto) }}" alt="{{ $kontak->nama }}" width="180" class="rounded mb-3">
                @endif
                <h5 class="card-title">{{ $kontak->nama }}</h5>
                <p class="card-text"><strong>Alamat:</strong> {{ $kontak->alamat }}</p>
                <p class="card-text"><strong>No HP:</strong> {{ $kontak->no_hp }}</p>
                <p class="card-text"><small class="text-muted">Ditambahkan {{ $kontak->created_at->diffForHumans() }}</small></p>
                <a href="{{ route('kontaks.edit', $kontak) }}" class="btn btn-warning">Edit</a>
                <form action="{{ route('kontaks.destroy', $kontak) }}" method="POST" class="d-inline">
                    @csrf
                    @method('DELETE')
                    <button type="submit" class="btn btn-danger" onclick="return confirm('Apakah Anda yakin ingin menghapus?')">Hapus</button>
                </form>
                <a href="{{ route('kontaks.index') }}" class="btn btn-secondary">Kembali</a>
            </div>
        </div>
    </div>
</div>
@endsection

```

3.10 Menambahkan Authentication Manual

Karena Breeze tidak dipasang, authentication ditulis sendiri pada AuthController. Bagian pertama menangani pendaftaran akun.

```php
<?php

namespace App\Http\Controllers;

use App\Models\User;
use Illuminate\Http\RedirectResponse;
use Illuminate\Http\Request;
use Illuminate\Support\Facades\Auth;
use Illuminate\View\View;

class AuthController extends Controller
{
    /**
     * Menampilkan form register.
     */
    public function showRegister(): View
    {
        return view('auth.register');
    }

    /**
     * Memvalidasi data, membuat akun baru, lalu langsung login.
     */
    public function register(Request $request): RedirectResponse
    {
        $validated = $request->validate([
            'name' => 'required|string|max:255',
            'email' => 'required|string|email|max:255|unique:users,email',
            'password' => 'required|string|min:8|confirmed', // Harus sama dengan password_confirmation
        ]);

        $user = User::create($validated); // Password otomatis di-hash oleh cast 'hashed' pada model User

        Auth::login($user);
        $request->session()->regenerate(); // Mencegah session fixation

        return redirect()->route('kontaks.index')->with(
            'success',
            'Registrasi berhasil, selamat datang ' . $user->name . '.'
        );
    }

```

Validasi register memakai min:8 dan confirmed, sehingga password minimal delapan karakter dan harus sama dengan kolom konfirmasi. Setelah User::create(), pengguna langsung login melalui Auth::login($user) dan session diregenerasi untuk mencegah session fixation.

Bagian kedua menangani login dan logout.

```php
    /**
     * Menampilkan form login.
     */
    public function showLogin(): View
    {
        return view('auth.login');
    }

    /**
     * Memeriksa email dan password, lalu membuat session login.
     */
    public function login(Request $request): RedirectResponse
    {
        $credentials = $request->validate([
            'email' => 'required|string|email',
            'password' => 'required|string',
        ]);

        if (Auth::attempt($credentials, $request->boolean('remember'))) {
            $request->session()->regenerate();

            return redirect()->intended(route('kontaks.index'))->with('success', 'Login berhasil.');
        }

        return back()
            ->withErrors(['email' => 'Email atau password salah.'])
            ->onlyInput('email');
    }

    /**
     * Mengakhiri session login.
     */
    public function logout(Request $request): RedirectResponse
    {
        Auth::logout();
        $request->session()->invalidate();      // Hapus seluruh data session
        $request->session()->regenerateToken(); // Buat token CSRF baru

        return redirect()->route('login')->with('success', 'Anda berhasil logout.');
    }
}

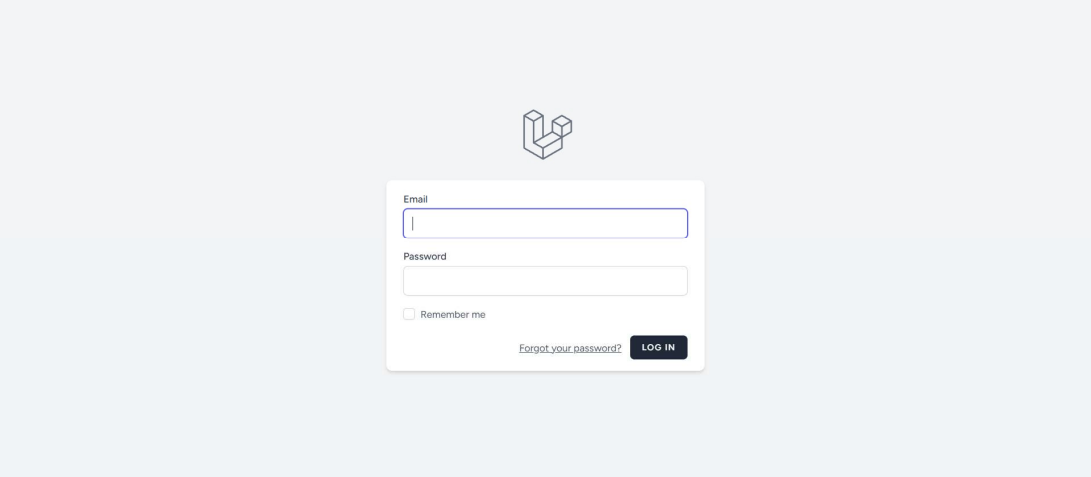
```

Auth::attempt() memeriksa email dan password. Jika berhasil, pengguna diarahkan ke halaman yang dituju sebelumnya atau ke daftar kontak. Jika gagal, form login dikembalikan dengan pesan "Email atau password salah." dan hanya kolom email yang diisi ulang. Method logout memanggil Auth::logout(), menghapus session, dan membuat token CSRF baru.

4. Hasil dan Dokumentasi Program

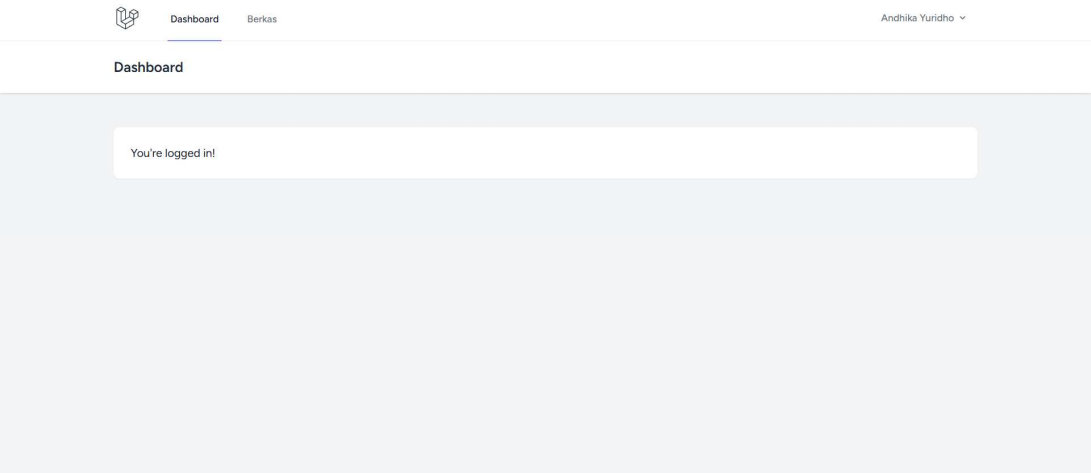
Bagian ini memuat tangkapan layar dari kedua aplikasi yang berjalan melalui php artisan serve. Versi Breeze diakses pada alamat http://127.0.0.1:8123 dan versi manual pada alamat

http://127.0.0.1:8124. Seluruh pengujian dikerjakan langsung melalui browser, dan isi database diperiksa melalui phpMyAdmin 5.2.3.



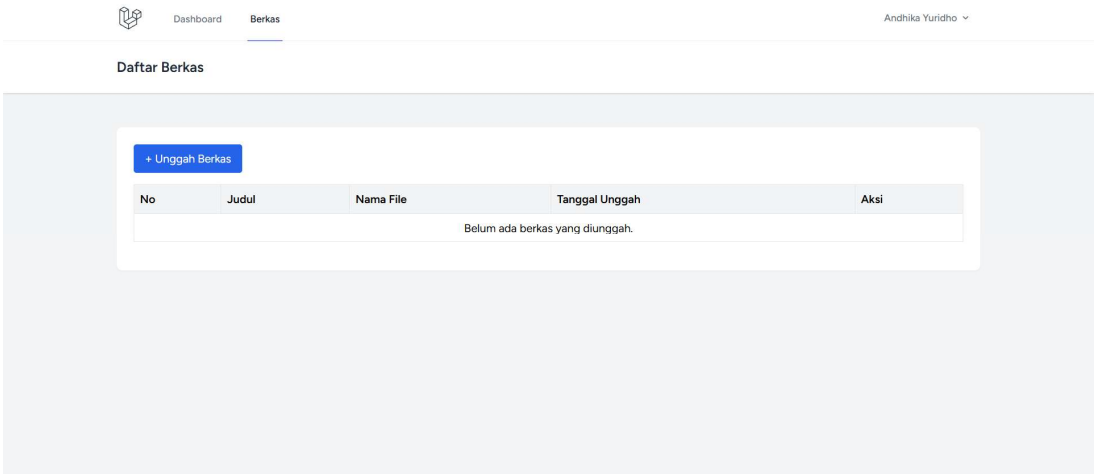
Gambar 1. Halaman login saat /uploads dibuka sebelum login

Membuka alamat /uploads tanpa session login langsung dialihkan middleware auth ke halaman /login. Halaman yang dituju tidak sempat tampil sama sekali.



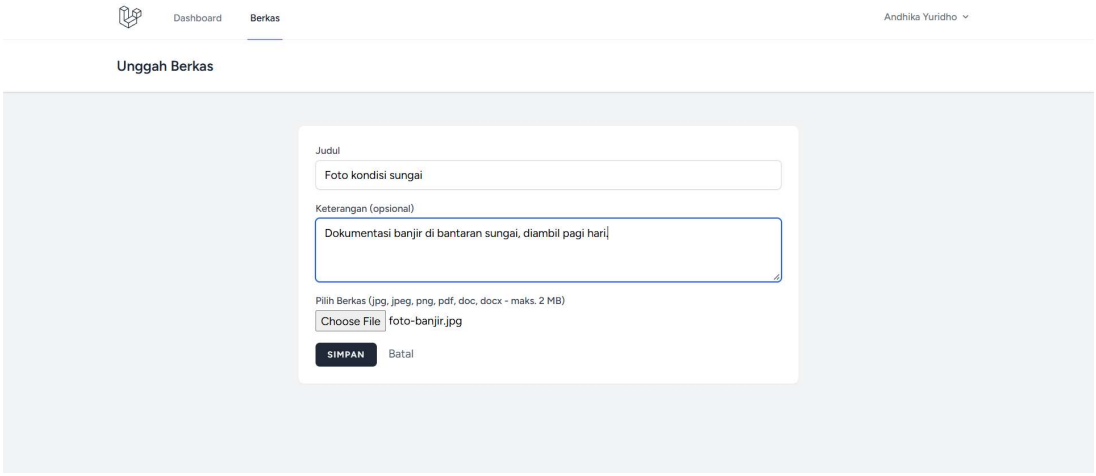
Gambar 2. Dashboard Breeze setelah login dengan menu Berkas

Setelah login, navigasi Breeze menampilkan menu Dashboard dan menu Berkas. Menu Berkas inilah pintu masuk ke seluruh halaman CRUD Berkas.



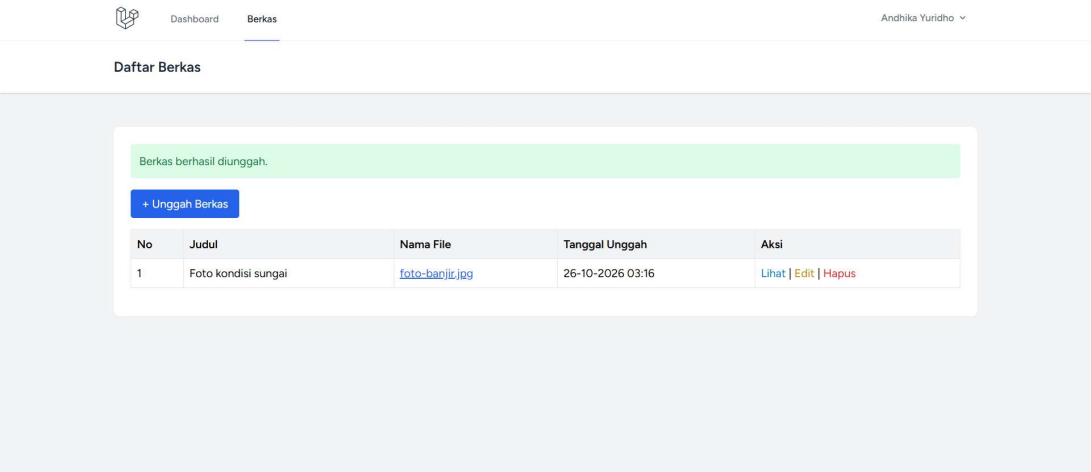
Gambar 3. Daftar Berkas masih kosong

Halaman index menampilkan pesan "Belum ada berkas yang diunggah." dari blok @empty, sedangkan tabel tetap tampil dengan header kolom No, Judul, Nama File, Tanggal Unggah, dan Aksi.



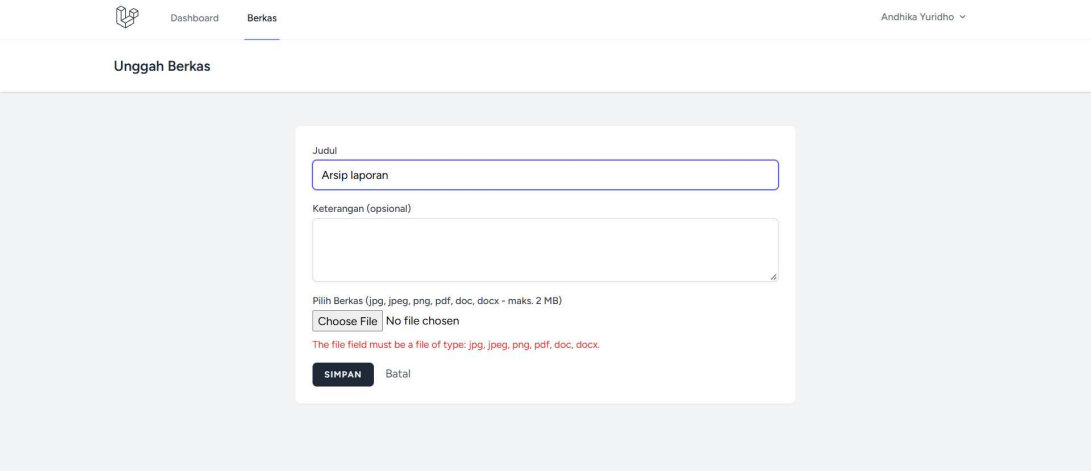
Gambar 4. Form Unggah Berkas yang sudah diisi

Form berisi input judul, keterangan, dan pemilih berkas. Berkas yang dipilih pada pengujian ini adalah gambar JPG berukuran 143 KB.



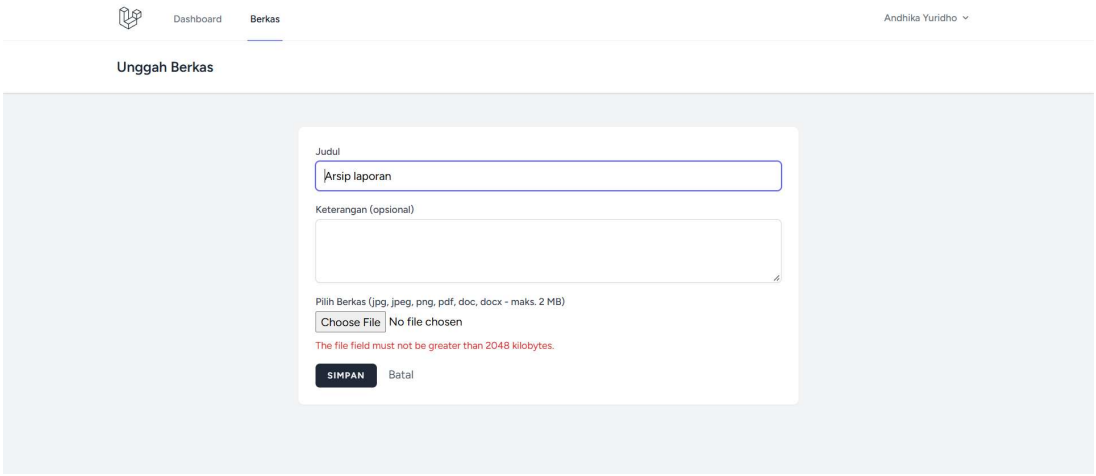
Gambar 5. Pesan berhasil setelah berkas diunggah

Setelah tombol Simpan ditekan, halaman kembali ke daftar berkas dengan pesan "Berkas berhasil diunggah." dan satu baris data baru. Kolom Nama File menampilkan nama asli berkas, bukan nama acak hasil store().



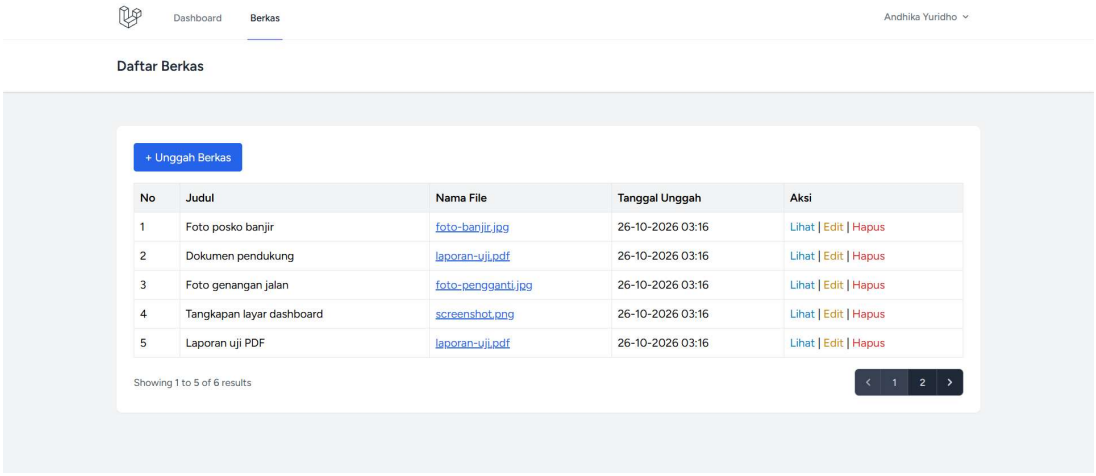
Gambar 6. Pesan galat saat mengunggah berkas ZIP

Berkas arsip.zip ditolak oleh aturan mimes. Pesan "The file field must be a file of type: jpg, jpeg, png, pdf, doc, docx." tampil tepat di bawah input berkas, dan isian judul tetap tersimpan.



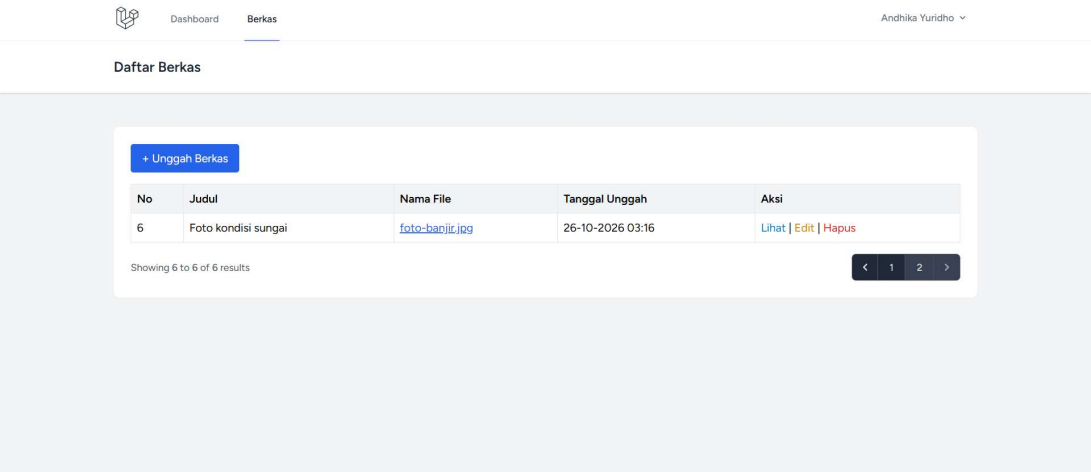
Gambar 7. Pesan galat saat mengunggah berkas 7,5 MB

Gambar berukuran 7,5 MB ditolak oleh aturan max:2048. Pesan yang muncul adalah "The file field must not be greater than 2048 kilobytes." Batas unggahan PHP pada mesin ini 2 GB, sehingga yang menolak berkas adalah validasi Laravel, bukan konfigurasi PHP.



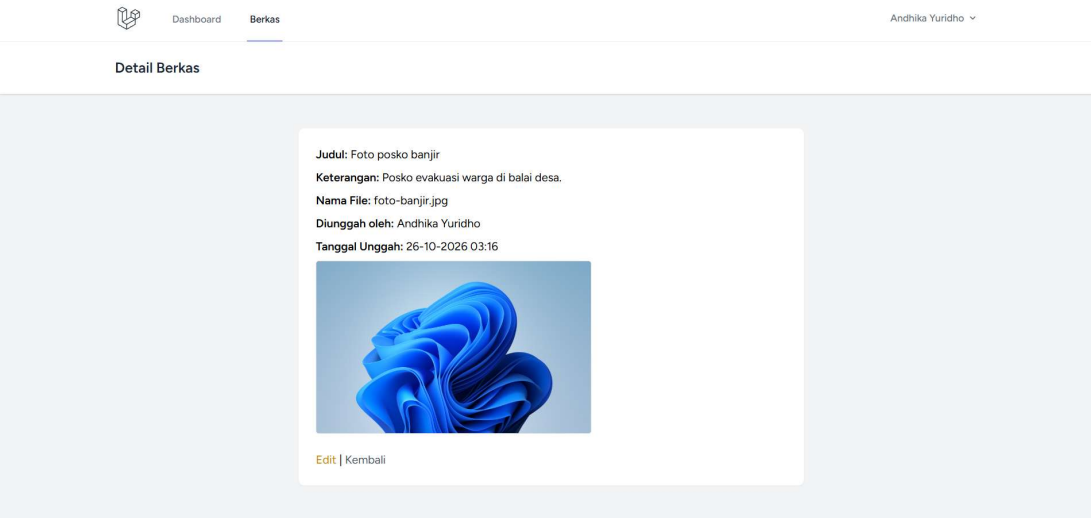
Gambar 8. Daftar Berkas halaman 1 dengan pagination

Enam berkas hasil pengujian ditampilkan lima per halaman. Nomor urut 1 sampai 5 muncul di halaman pertama, dan keterangan di bawah tabel menyebut Showing 1 to 5 of 6 results.



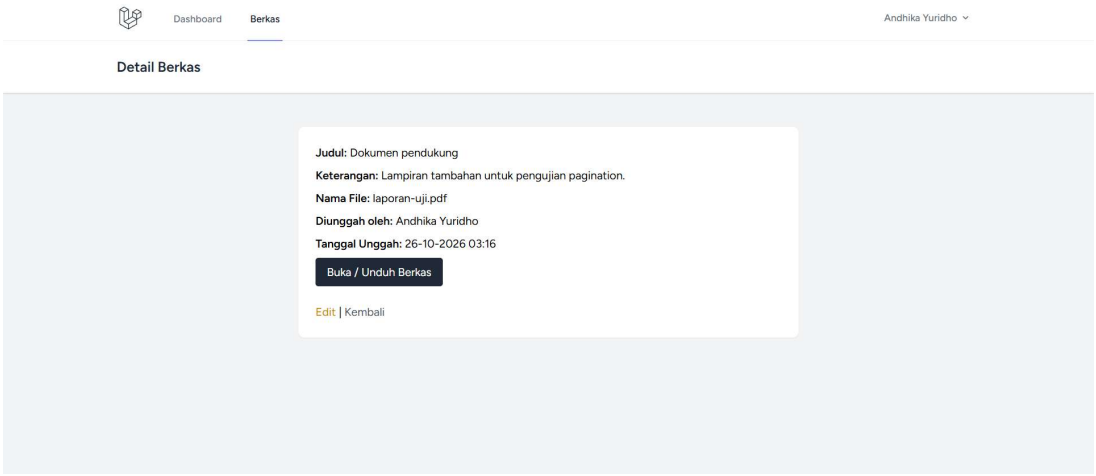
Gambar 9. Daftar Berkas halaman 2

Halaman kedua hanya berisi satu berkas dengan nomor urut 6, jadi penomoran tetap berlanjut karena memakai firstItem() ditambah indeks perulangan.



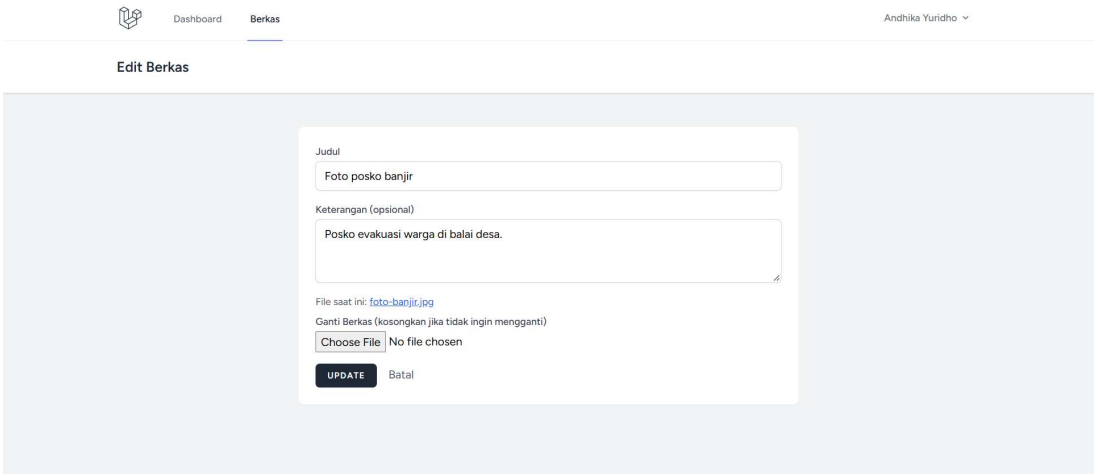
Gambar 10. Halaman detail berkas gambar

Halaman detail menampilkan judul, keterangan, nama file, nama pemilik dari relasi user, tanggal unggah, dan pratinjau gambar yang diambil dari URL storage.



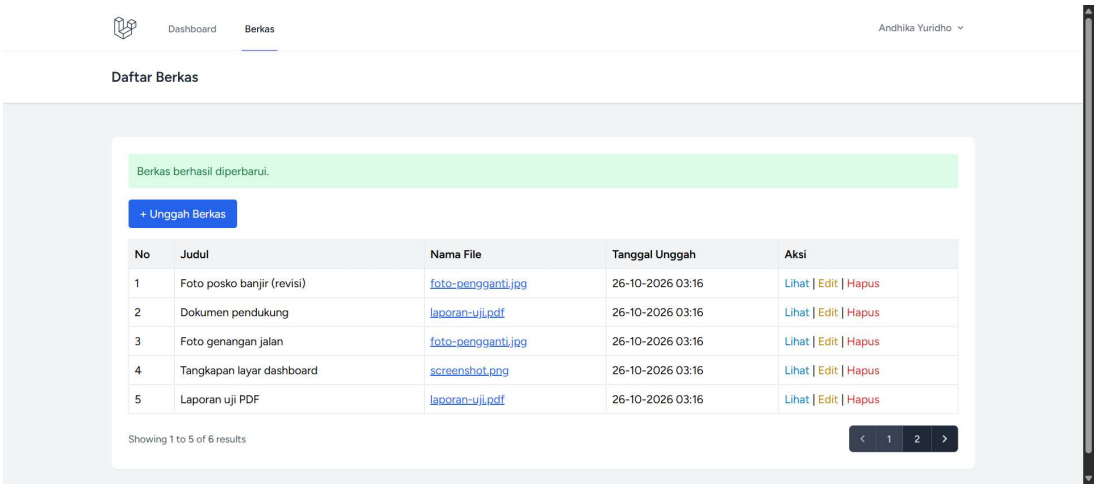
Gambar 11. Halaman detail berkas PDF

Untuk berkas PDF, pratinjau gambar tidak ditampilkan. Halaman menampilkan tombol Buka / Unduh Berkas yang membuka file di tab baru.



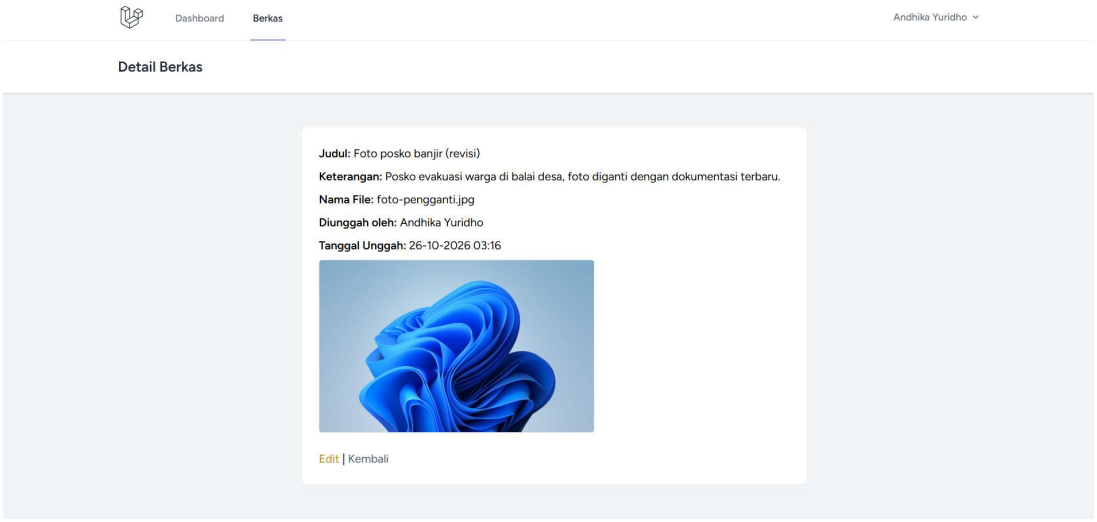
Gambar 12. Form Edit Berkas dengan tautan file saat ini

Form edit menampilkan data lama pada input judul dan keterangan, ditambah tautan foto-banjir.jpg sebagai berkas yang sedang tersimpan. Input penggantian berkas tidak wajib diisi.



Gambar 13. Pesan berhasil setelah berkas diperbarui

Judul diubah menjadi Foto posko banjir (revisi) dan berkas diganti dengan foto lain. Pesan "Berkas berhasil diperbarui." tampil di atas tabel, dan kolom Nama File berubah menjadi foto-pengganti.jpg.

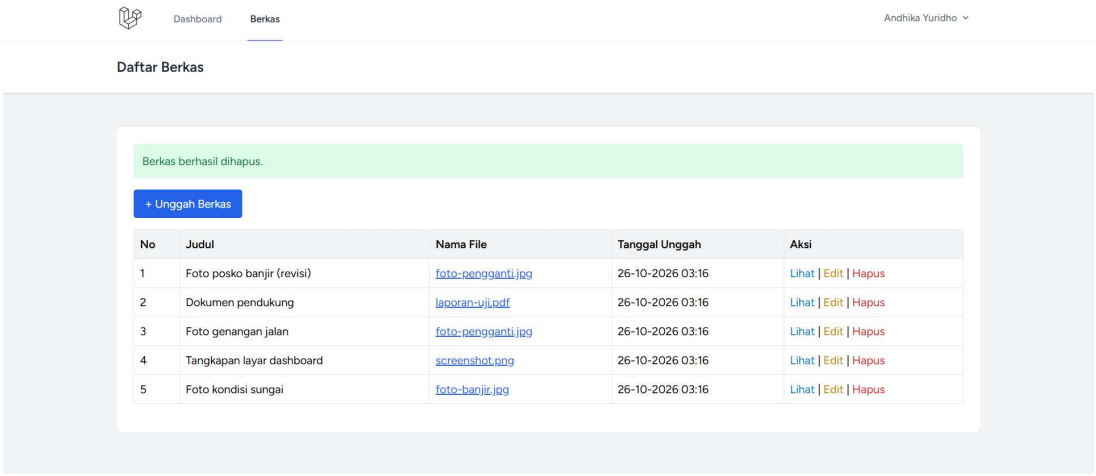


Gambar 14. Detail berkas setelah file diganti

Pratinjau menampilkan gambar pengganti. Berkas lama sudah tidak ada lagi di folder storage karena dihapus pada method update sebelum path baru disimpan.

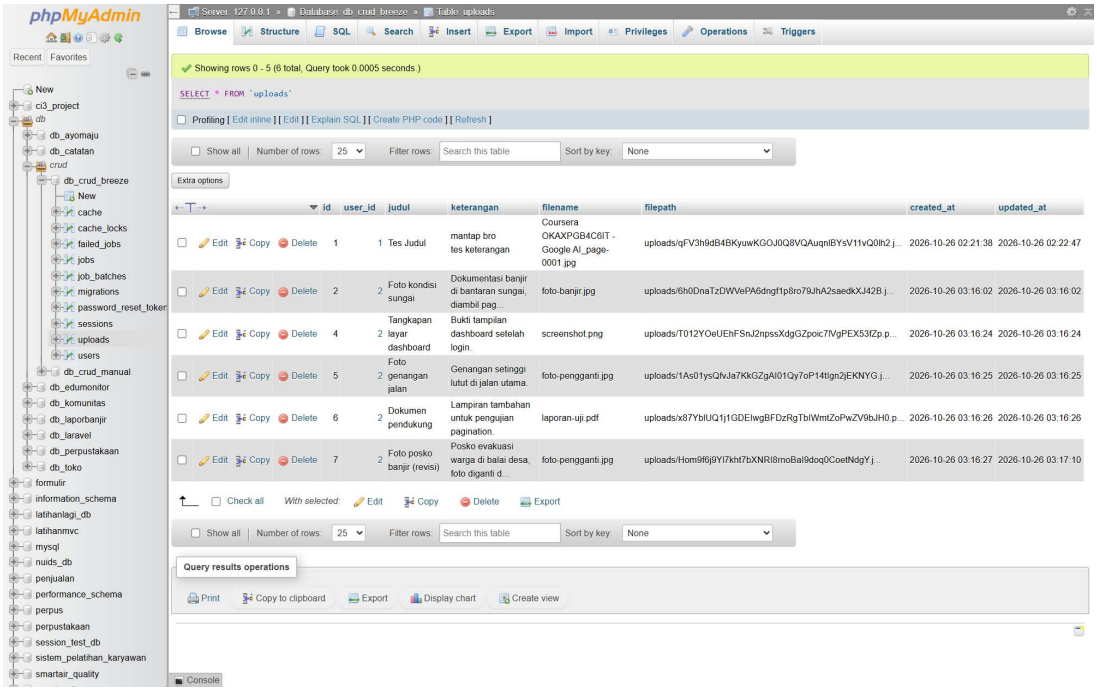
Gambar 15. Galat 403 saat membuka berkas milik pengguna lain

Alamat /uploads/1 dibuka memakai akun yang bukan pemilik berkas tersebut. abort_if() pada method show menghentikan request dan halaman menampilkan 403 Forbidden.



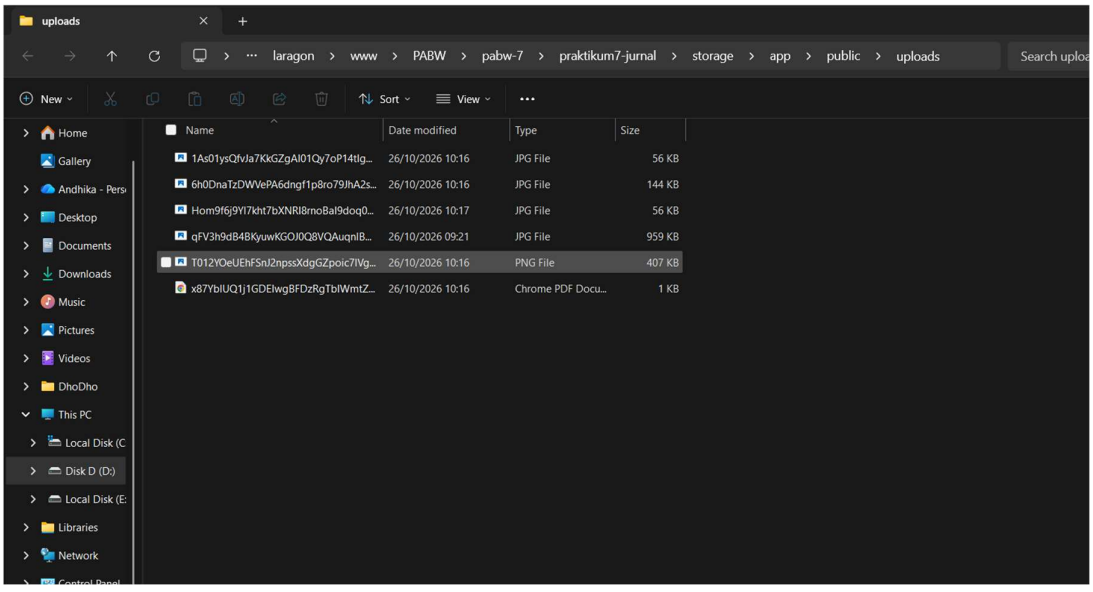
Gambar 16. Pesan berhasil setelah berkas dihapus

Setelah konfirmasi penghapusan diterima, baris data dan file fisiknya ikut terhapus. Pesan "Berkas berhasil dihapus." tampil di atas tabel dan jumlah data berkurang menjadi lima.



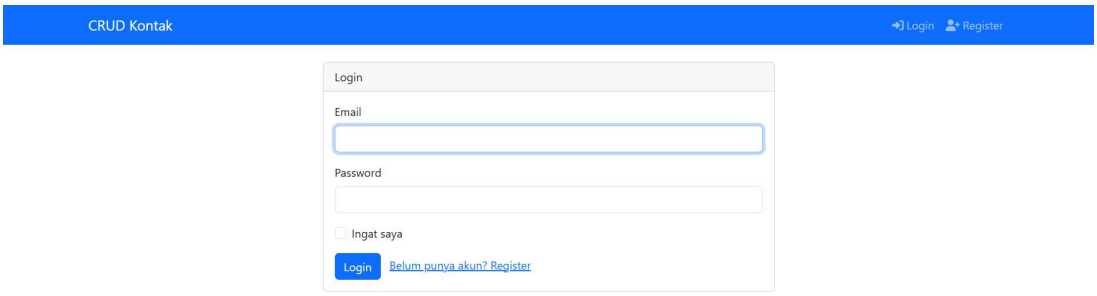
Gambar 17. Isi tabel uploads pada phpMyAdmin

Tabel uploads menyimpan id, user_id, judul, keterangan, filename, filepath, dan timestamp. Kolom filepath berisi nama acak seperti uploads/x87YblUQ1j1GDEIwgBFDzRgTbIWmtZoPwZV9bJH0.pdf, sedangkan kolom filename menyimpan nama asli berkas.



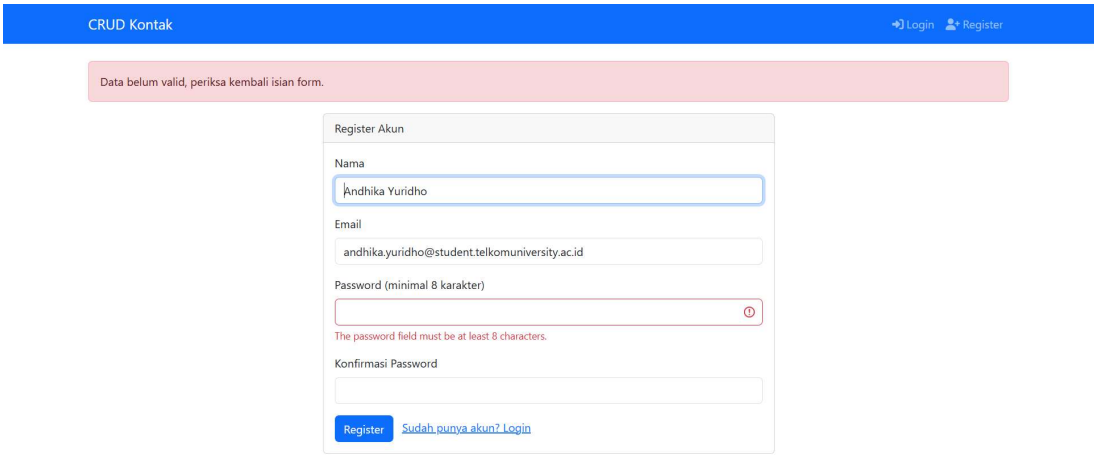
Gambar 18. Isi folder storage/app/public/uploads

Folder uploads berisi enam berkas dengan nama acak, sama banyak dengan jumlah baris pada tabel uploads. Tidak ada berkas sisa dari proses penggantian maupun penghapusan.



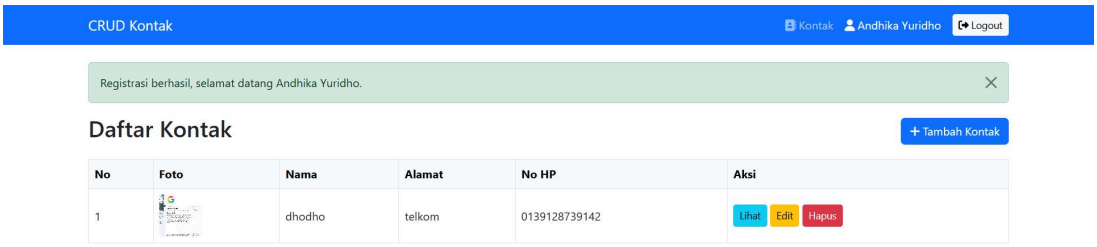
Gambar 19. Halaman /kontaks sebelum login

Halaman daftar kontak tidak bisa dibuka tanpa login. Middleware auth mengalihkan pengguna ke halaman login, dan navbar hanya menampilkan menu Login dan Register.



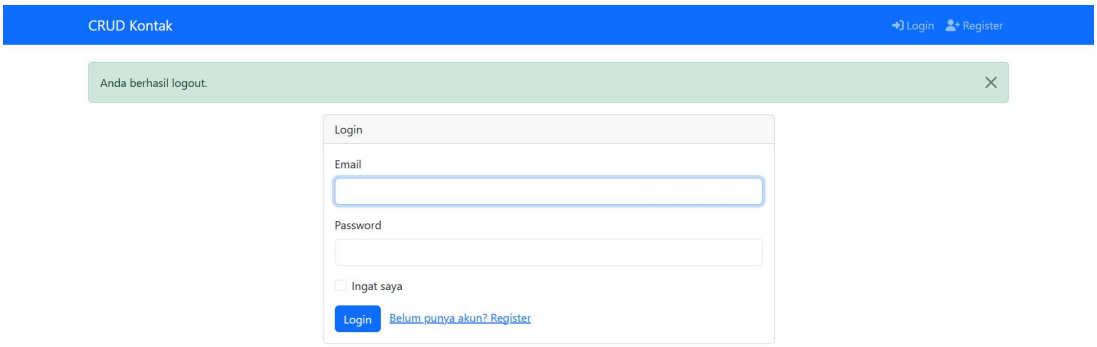
Gambar 20. Pesan galat validasi pada form Register

Password diisi tiga karakter dan konfirmasinya berbeda. Pesan The password field must be at least 8 characters. tampil di bawah input password, sedangkan nama dan email tetap terisi karena memakai old().



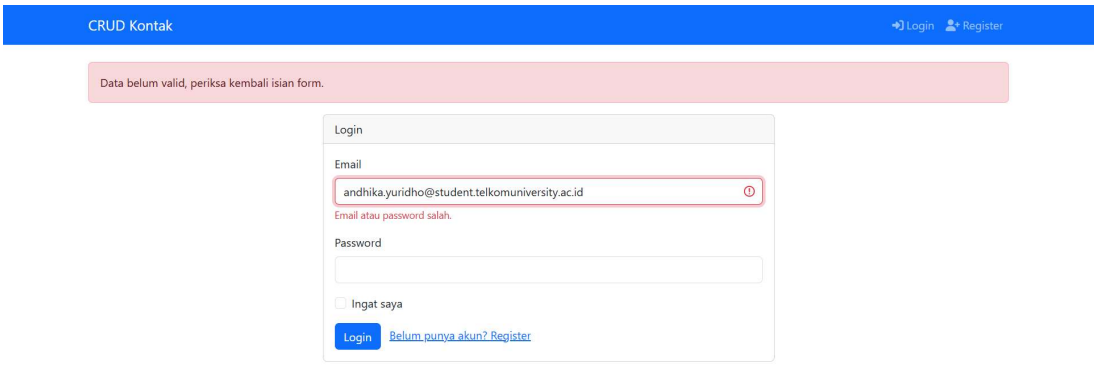
Gambar 21. Berhasil register dan langsung masuk ke daftar kontak

Setelah register berhasil, pengguna langsung login dan diarahkan ke daftar kontak. Navbar berubah menampilkan nama pengguna dan tombol Logout, disertai pesan "Registrasi berhasil, selamat datang Andhika Yuridho."



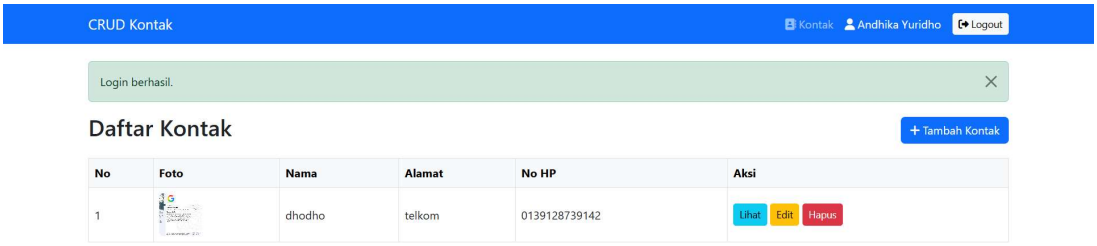
Gambar 22. Pesan berhasil setelah logout

Logout mengakhiri session dan mengembalikan pengguna ke halaman login dengan pesan "Anda berhasil logout." Halaman kontak tidak dapat dibuka lagi sebelum login.



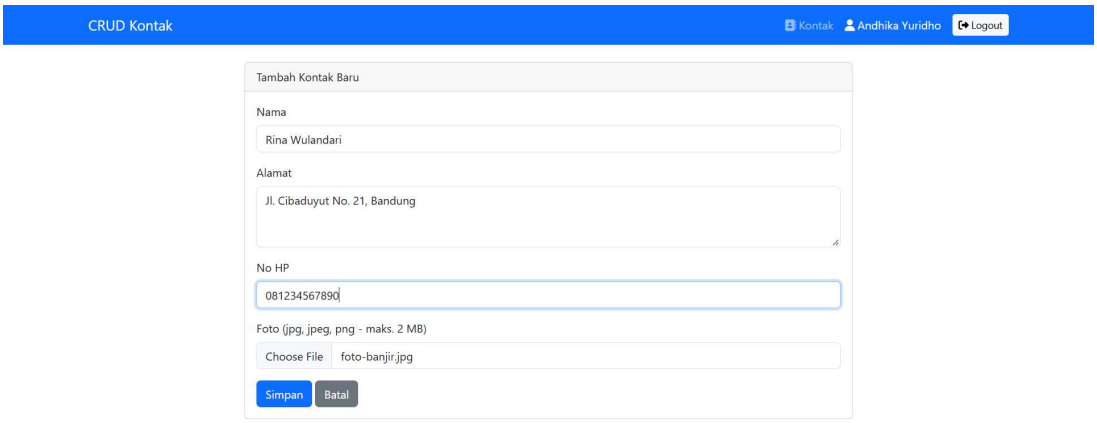
Gambar 23. Pesan galat saat login dengan password salah

Auth::attempt() mengembalikan false sehingga halaman login ditampilkan lagi dengan pesan "Email atau password salah." Hanya kolom email yang diisi ulang.



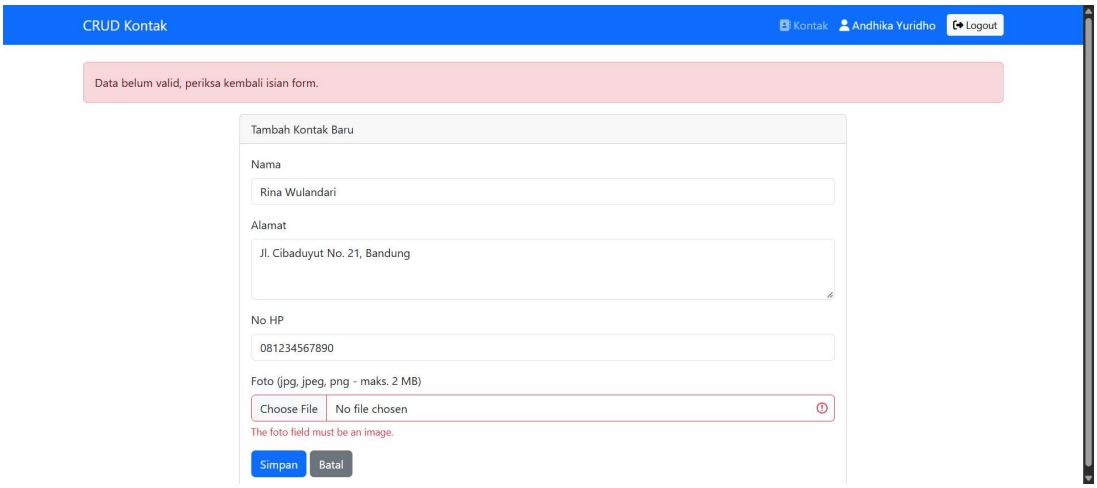
Gambar 24. Berhasil login dan kembali ke daftar kontak

Login dengan email dan password yang benar menampilkan pesan "Login berhasil." dan daftar kontak beserta tombol pengelolaannya dapat dipakai kembali.



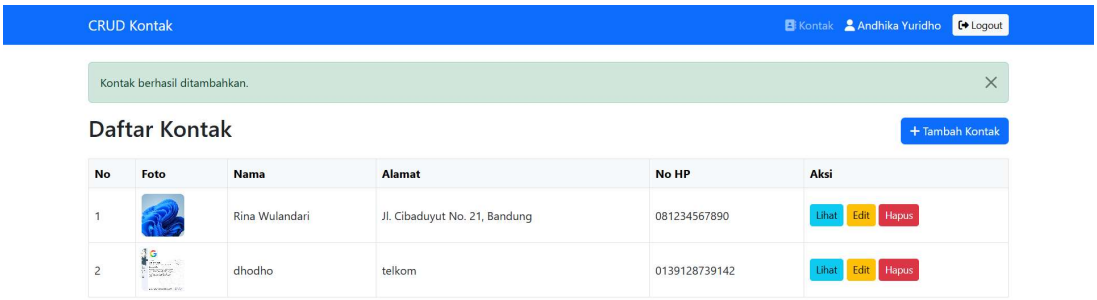
Gambar 25. Form Tambah Kontak yang sudah diisi

Form berisi input nama, alamat, no HP, dan foto. Foto pada pengujian ini berukuran 143 KB dan dipilih dari berkas lokal.



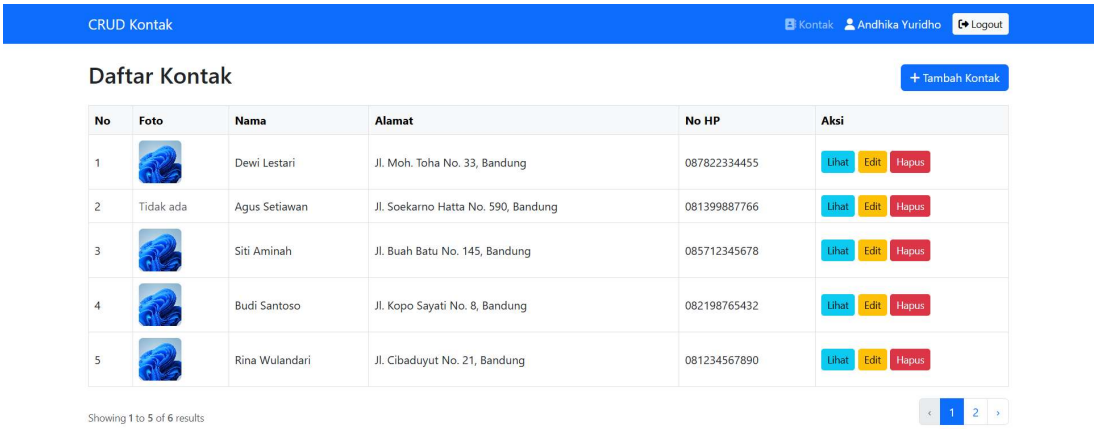
Gambar 26. Pesan galat saat foto yang dipilih bukan gambar

Berkas ZIP diunggah pada input foto. Aturan image dan mimes menolaknya dengan pesan "The foto field must be an image." Input foto diberi garis merah dan ringkasan galat muncul di atas form.



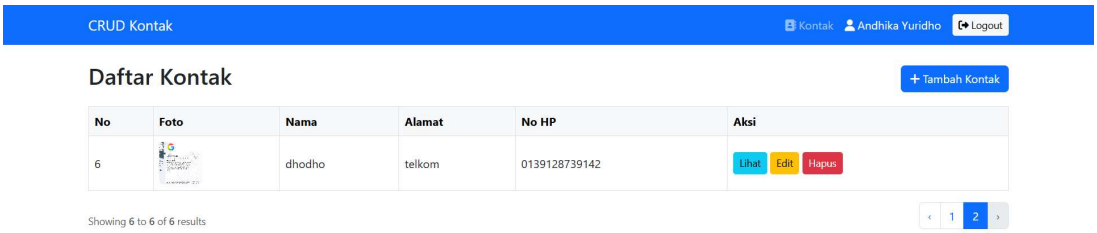
Gambar 27. Pesan berhasil setelah kontak ditambahkan

Kontak baru muncul di baris pertama daftar dengan thumbnail foto. Pesan "Kontak berhasil ditambahkan." tampil di atas tabel.



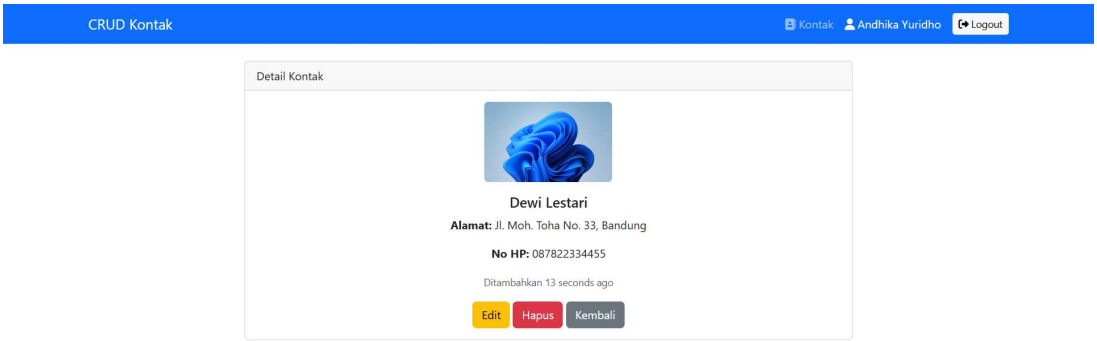
Gambar 28. Daftar Kontak halaman 1 dengan pagination Bootstrap

Enam kontak ditampilkan lima per halaman. Kontak tanpa foto menampilkan teks Tidak ada pada kolom Foto, dan tombol pagination memakai gaya Bootstrap karena Paginator::useBootstrapFive() sudah aktif.



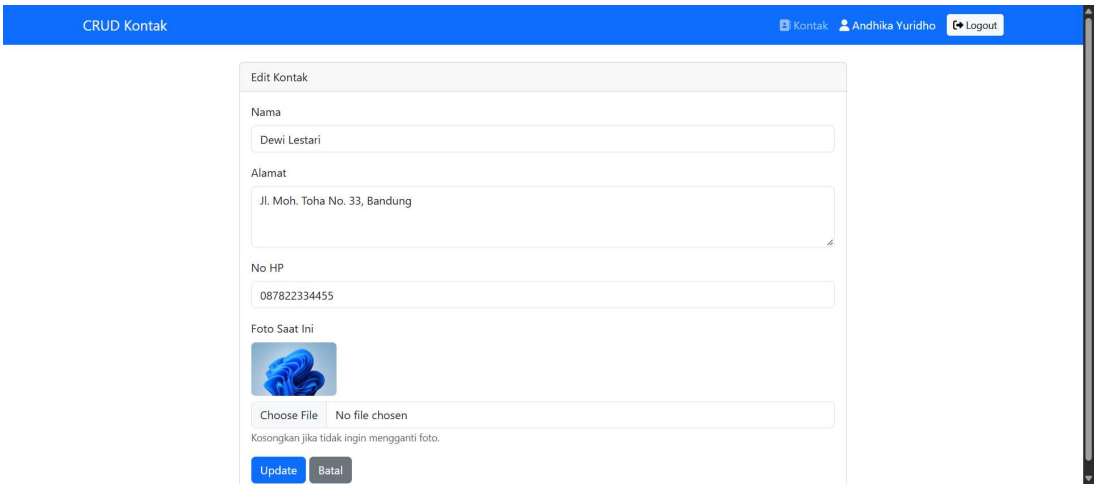
Gambar 29. Daftar Kontak halaman 2

Halaman kedua berisi satu kontak dengan nomor urut 6. Penomoran tetap berlanjut karena memakai firstItem() ditambah indeks perulangan.



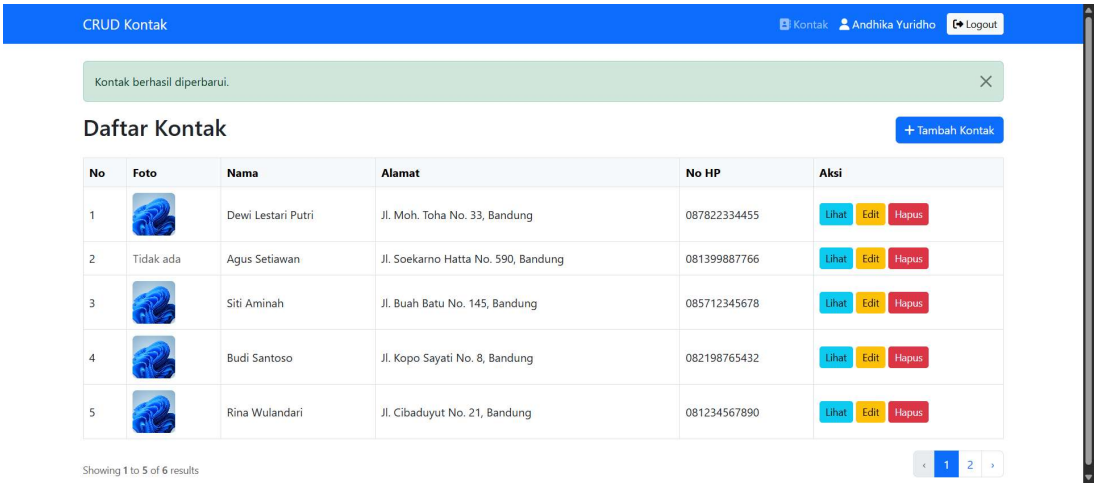
Gambar 30. Halaman detail kontak

Halaman detail menampilkan foto berukuran 180 piksel, nama, alamat, nomor HP, dan waktu penambahan dari diffForHumans(), misalnya "Ditambahkan 13 seconds ago".



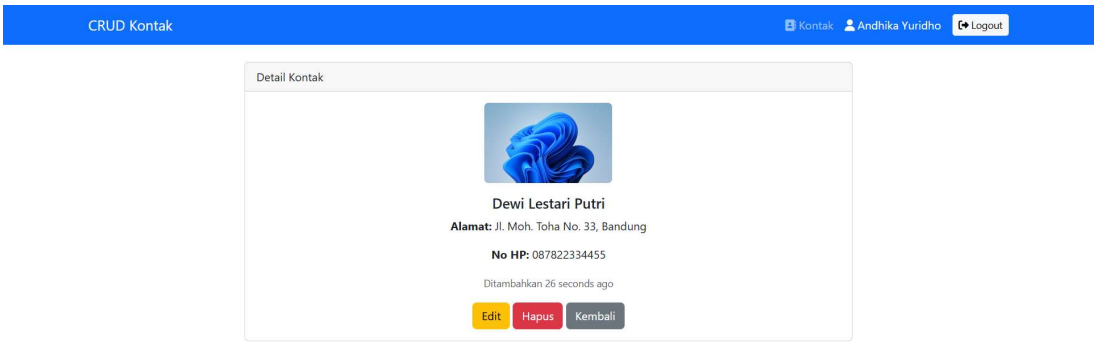
Gambar 31. Form Edit Kontak dengan foto saat ini

Form edit menampilkan data lama dan foto yang sedang tersimpan. Input foto diberi keterangan bahwa kolom tersebut boleh dikosongkan jika foto tidak ingin diganti.



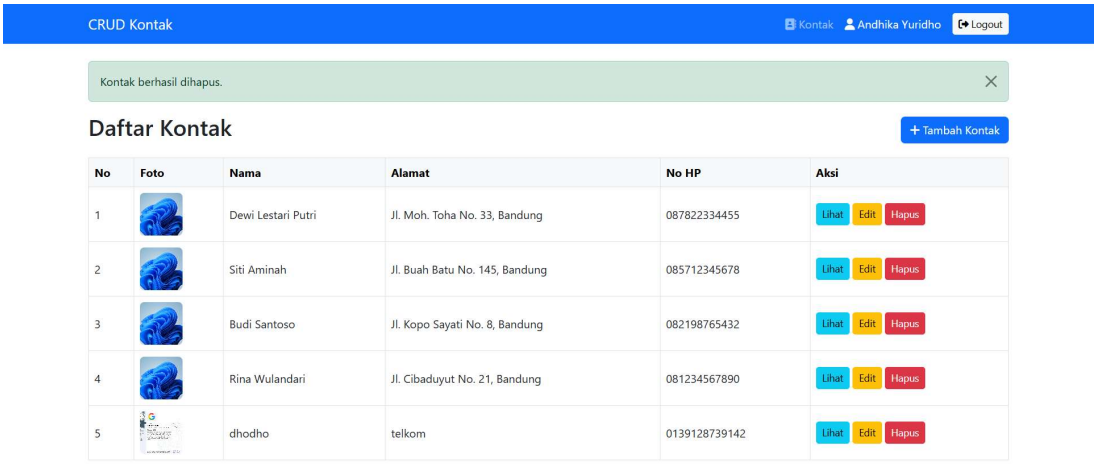
Gambar 32. Pesan berhasil setelah kontak diperbarui

Nama diubah menjadi Dewi Lestari Putri dan fotonya diganti. Pesan "Kontak berhasil diperbarui." tampil di atas tabel, dan thumbnail pada baris pertama ikut berubah.



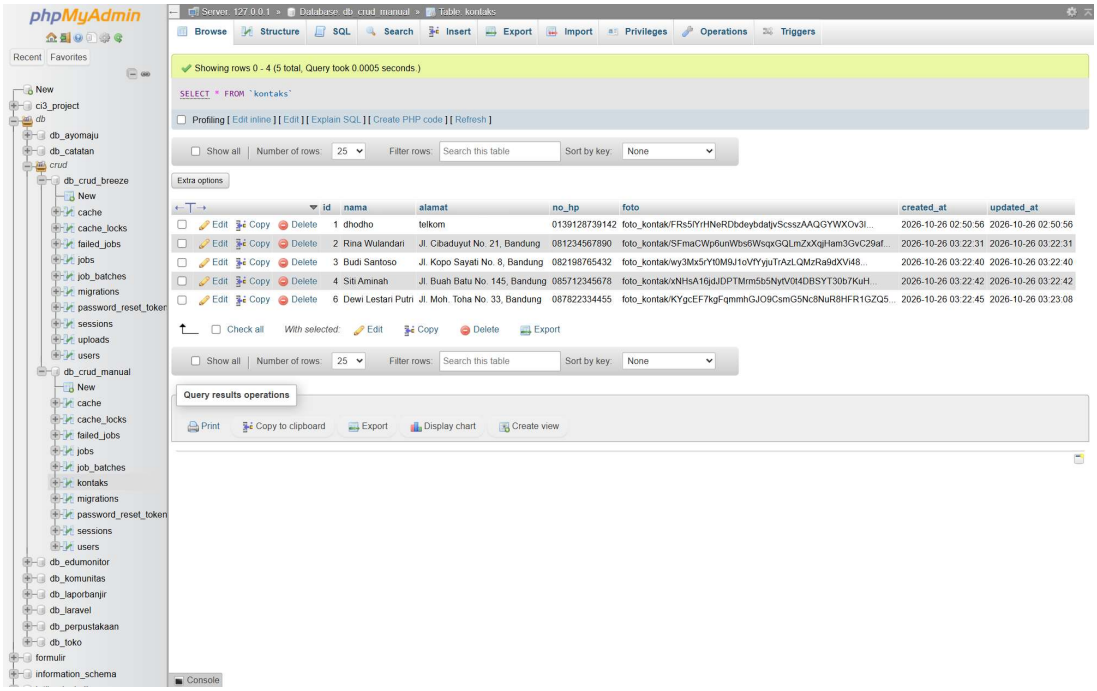
Gambar 33. Detail kontak setelah foto diganti

Halaman detail menampilkan nama baru dan foto pengganti. Foto lama sudah dihapus dari folder storage pada saat proses update.



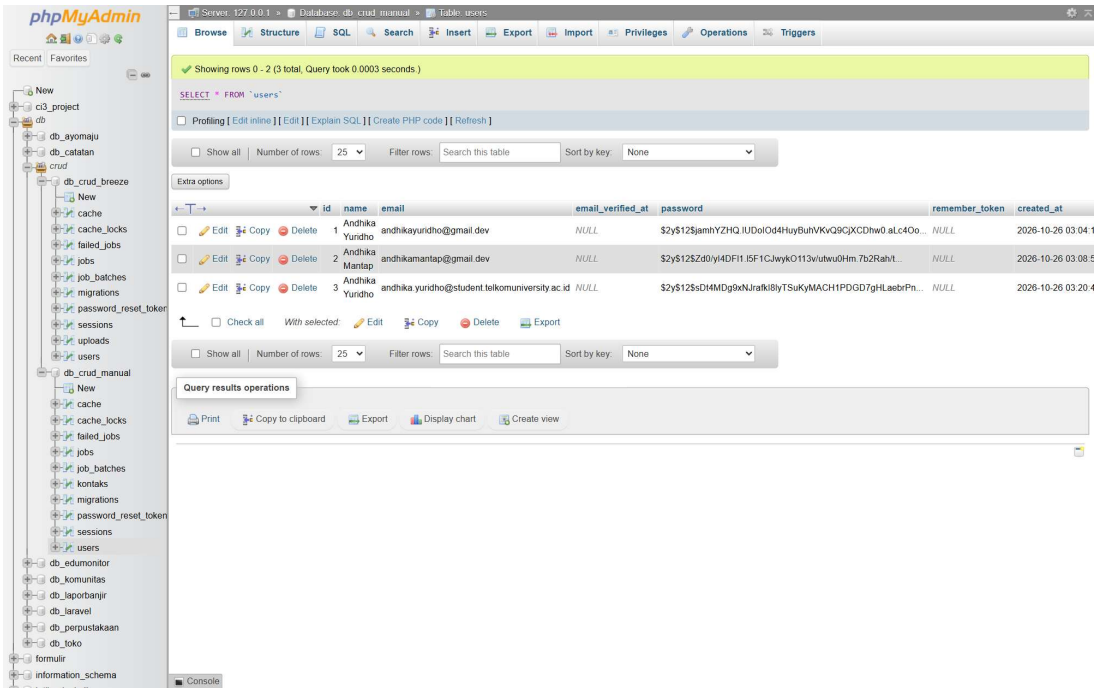
Gambar 34. Pesan berhasil setelah kontak dihapus

Kontak tanpa foto dihapus setelah konfirmasi. Pesan "Kontak berhasil dihapus." tampil di atas tabel dan jumlah data berkurang menjadi lima.



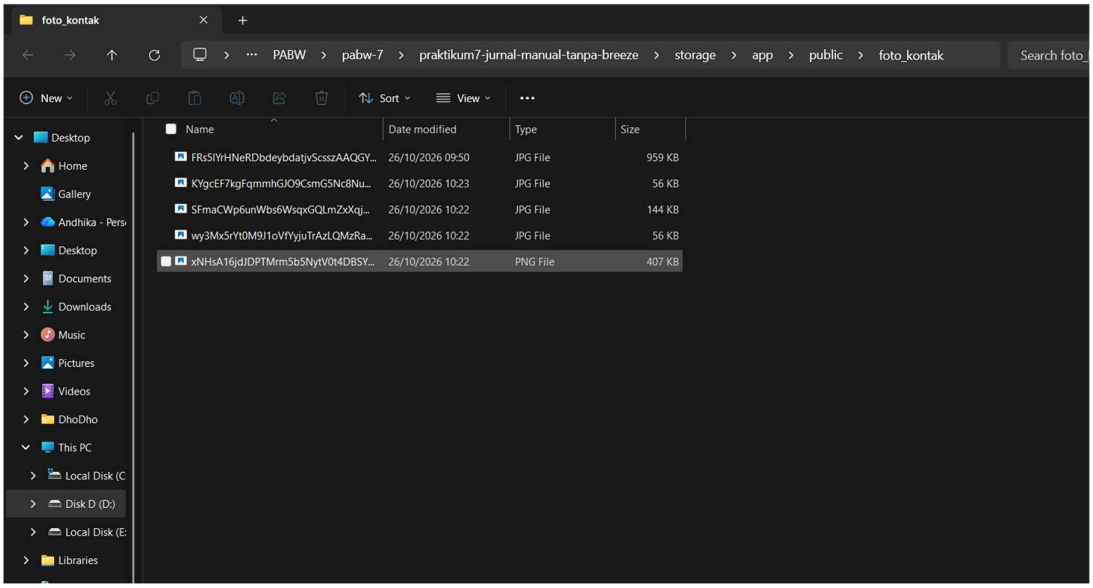
Gambar 35. Isi tabel kontaks pada phpMyAdmin

Tabel kontaks menyimpan nama, alamat, no_hp, foto, dan timestamp. Kolom foto berisi path seperti foto_kontak/FRs5lYrHNeRDbdeybdatjvScsszAAQGYWXOv3IMg.jpg, bukan isi gambarnya.



Gambar 36. Kolom password pada tabel users berisi hash bcrypt

Password akun tersimpan sebagai hash yang diawali $2y$, sehingga password asli tidak bisa dibaca langsung dari database. Hash ini dihasilkan oleh cast hashed pada model User.



Gambar 37. Isi folder storage/app/public/foto_kontak

Folder foto_kontak berisi lima berkas gambar dengan nama acak, sama dengan jumlah kontak yang memiliki foto setelah satu kontak dihapus.

5. Analisis dan Pembahasan

Resource controller dan resource route memangkas banyak pekerjaan yang pada modul sebelumnya ditulis satu per satu. Satu baris Route::resource('uploads', UploadController::class) menghasilkan tujuh route dengan nama dan method HTTP yang sudah baku, dan controller hasil make:controller --resource sudah menyediakan tujuh method kosong yang tinggal diisi. Route model binding juga ikut bekerja: parameter Upload $upload pada method show, edit, update, dan destroy otomatis diisi data berdasarkan id pada URL, sehingga tidak perlu menulis Upload::find($id) dan menangani kasus data tidak ditemukan secara manual.

Pembatasan akses pada versi Breeze bergantung pada dua lapis. Lapisan pertama adalah middleware auth pada grup route, yang menahan pengguna yang belum login dan mengarahkannya ke halaman login seperti terlihat pada Gambar 1. Lapisan kedua adalah abort_if($upload->user_id !== Auth::id(), 403) di dalam controller, yang menahan pengguna yang sudah login tetapi bukan pemilik berkas. Tanpa lapisan kedua, siapa pun yang sudah login bisa membuka /uploads/1 dan melihat berkas milik orang lain hanya dengan menebak

id. Pengujian pada Gambar 15 menunjukkan lapisan ini bekerja dan menghasilkan 403 Forbidden.

Pemisahan antara isi berkas dan path di database terlihat jelas pada hasil pengujian. Tabel uploads pada Gambar 17 hanya menyimpan nama asli berkas dan path hasil store(), sedangkan isi berkasnya berada di storage/app/public/uploads seperti pada Gambar 18. Nama berkas di storage berupa string acak, sehingga dua berkas dengan nama sama dari dua pengguna berbeda tidak akan saling menimpa. Cara ini juga membuat ukuran database tetap kecil dan berkas bisa dipindahkan tanpa mengubah struktur tabel. Konsekuensinya, folder public/storage harus ada sebagai symbolic link ke storage/app/public, karena folder storage sendiri berada di luar document root dan tidak bisa diakses langsung dari browser.

Konsistensi antara database dan storage dijaga di dua tempat. Method update menghapus berkas lama dengan Storage::disk('public')->delete($upload->filepath) sebelum menyimpan path baru, dan method destroy menghapus berkas fisik sebelum menghapus barisnya. Setelah satu berkas diganti dan satu berkas dihapus, tabel uploads berisi enam baris dan folder uploads berisi enam berkas, termasuk satu baris dan satu berkas milik akun yang dipakai sebelum pengujian ini. Tidak ada berkas yatim yang tertinggal. Pada versi manual, pola yang sama diterapkan pada kolom foto di tabel kontaks, dengan lima baris dan lima berkas setelah satu kontak dihapus.

Validasi file menentukan berkas mana yang boleh masuk ke server. Aturan mimes:jpg,jpeg,png,pdf,doc,docx menolak arsip ZIP seperti pada Gambar 6, dan aturan max:2048 menolak gambar 7,5 MB seperti pada Gambar 7. Pesan galat yang muncul adalah pesan bawaan Laravel dalam bahasa Inggris, bukan pesan yang saya tulis sendiri, karena pada kedua project tidak ada file bahasa atau argumen pesan khusus pada method validate(). Pada mesin ini upload_max_filesize dan post_max_size bernilai 2 GB, jadi berkas 7,5 MB tetap sampai ke aplikasi dan ditolak oleh validator, bukan oleh PHP.

Perbedaan Breeze dan versi manual paling terasa pada lapisan tampilan dan authentication, bukan pada logika CRUD. Versi Breeze memakai komponen x-app-layout, x-text-input, dan x-input-error yang sudah tersedia, serta memerlukan Node.js dan proses build agar kelas Tailwind yang dipakai pada view ikut terkompilasi. Versi manual memakai layout Blade sendiri dengan Bootstrap dari CDN, sehingga tidak perlu Node.js, tetapi seluruh navbar, tampilan pesan, dan penanda galat harus ditulis manual memakai @error dan kelas is-invalid. Authentication juga berbeda: Breeze sudah menyediakan route, controller, dan view untuk

register, login, dan logout, sedangkan versi manual memerlukan AuthController sendiri dengan Auth::attempt(), regenerate session, dan middleware auth pada route. Bagian yang identik adalah resource controller, validasi, store() ke disk public, dan Storage::delete() saat berkas diganti atau dihapus.

Beberapa kendala muncul selama pengerjaan dan tercatat pada file storage/logs/laravel.log. Pertama, pada 2026-10-26 02:15:10 muncul galat Undefined constant "uploads" ketika view uploads/index.blade.php dirender. Pesan galat menunjuk baris 27 pada file view hasil kompilasi. Saya memperbaiki penulisan pada baris tersebut, dan setelah itu halaman daftar berkas tampil normal tanpa entri galat baru di log.

Kedua, pada 2026-10-26 02:49:02 muncul galat SQLSTATE[42S02] Table 'db_crud_manual.kontaks' doesn't exist saat halaman daftar kontak dibuka. Penyebabnya migration tabel kontaks belum dijalankan pada database db_crud_manual. Setelah php artisan migrate dijalankan, galat hilang dan tabel kontaks terbentuk.

Ketiga, tombol + Unggah Berkas pada halaman daftar berkas tidak terlihat meskipun teksnya ada di dalam HTML. Saya memeriksa elemen tersebut di browser dan menemukan background-color-nya transparan, artinya kelas bg-blue-600 tidak ada pada file CSS hasil build. Folder public/build masih berisi CSS yang dikompilasi sebelum view uploads dibuat. Setelah npm run build dijalankan ulang, background-color tombol berubah menjadi rgb(37, 99, 235) dan tombol tampil normal seperti pada Gambar 3.

Keempat, pada file resources/views/auth/register.blade.php terdapat kelas invalid-\feedback pada penampil pesan galat password. Kelas itu tidak dikenal Bootstrap, sehingga pesan galat password tampil tanpa gaya merah dan tanpa border pada input. Setelah ditulis menjadi invalid-feedback, pesan galat password tampil seperti input lain pada Gambar 20.

Kelima, alamat http://localhost/phpmyadmin mengembalikan 403 Forbidden pada instalasi Laragon yang saya pakai. Virtual host phpMyAdmin berada pada alamat http://phpmyadmin.test, dan seluruh pemeriksaan database pada Gambar 17, 35, dan 36 dilakukan dari alamat tersebut.

6. Kesimpulan

Praktikum ini menghasilkan dua aplikasi CRUD dengan upload file yang berjalan. Versi Breeze mengelola berkas milik pengguna yang sedang login, lengkap dengan pagination, pratinjau gambar, penggantian berkas, penghapusan berkas beserta file fisiknya, dan

penolakan akses 403 untuk berkas milik pengguna lain. Versi manual mengelola kontak dengan foto memakai layout Bootstrap dan authentication yang ditulis sendiri.

Saya memahami bahwa resource controller dan resource route hanya menyediakan kerangka route dan method, sedangkan isi tiap method tetap ditulis sendiri. Berkas disimpan di storage/app/public dan database hanya menyimpan path-nya, sehingga symbolic link public/storage wajib ada agar berkas bisa ditampilkan. Validasi jenis dan ukuran berkas harus dilakukan sebelum store(), dan berkas lama perlu dihapus setiap kali berkas diganti atau datanya dihapus agar storage tidak menumpuk file yang tidak terpakai.

Perbedaan Breeze dan pengerjaan manual terletak pada tampilan dan authentication. Logika CRUD dan upload file pada kedua cara sama persis, yaitu validasi, store() ke disk public, dan Storage::delete() untuk membersihkan berkas lama.

7. Link Repository Github

https://github.com/dhodhoo/pabw-andhika-yuridho/tree/main/pabw-7